
Entwicklung einer Android-App zur Akkuvorhersage mit TinyML

Portfolio-Dokumentation

Vertiefung I: Mobile und Ubiquitäre Anwendungen

Vorgelegt von: Keno Schürger

Matrikelnummer: 5023033

Studiengang: Wirtschaftsinformatik (BWI)

Dozent: Prof. Dr. Karsten Huffstadt

Semester: Sommersemester 2026



App direkt installieren
(Android, Sideload via Browser)

Hinweis zur App. Wer die App ohne Bauen-aus-dem-Quellcode direkt installieren möchte, kann den QR-Code scannen – das lädt die signierte Debug-APK (`HandyUsage-2026-05.apk`, ca. 22 MB) direkt aus dem GitHub-Repo aufs Handy. Beim ersten Start fragt Android nach der Erlaubnis zur Installation aus unbekannter Quelle. Nach dem Onboarding sammelt die App alle 30 Sekunden Sensor-Daten. Eine erste eigene Vorhersage erscheint nach 5 Minuten (zehn Messpunkte). Der Algorithmus-Tab funktioniert sofort und zeigt die Architektur des mitgelieferten Modells.

Würzburg

Inhaltsverzeichnis

Abbildungsverzeichnis	III
Tabellenverzeichnis	III
0 Ausgangssituation und Zielsetzung	1
0.1 Alleinstellungsmerkmal	1
0.2 Problemstellung	1
0.3 Konkrete Anwendungsszenarien	2
0.4 Mehrwert aus Nutzer- und Marktsicht	2
0.5 Zielsetzung	3
0.6 Rahmenbedingungen	4
0.7 Eckdaten des Projekts	4
1 Architekturmodell	5
1.1 Was passiert im Inneren – in einfachen Worten	5
1.2 Gesamtarchitektur im Überblick	6
1.3 Android-App im Detail	8
1.4 Die App in Aktion – Bedienungsanleitung mit Screenshots	9
1.5 Python-Pipeline im Detail	18
2 Datenmodell	19
2.1 CSV-Schema	19
2.2 Warum genau diese zehn Features?	19
2.3 Speicherung und Export	20
2.4 Datenschutz	20
3 Programmierung	23
3.1 Datensammlung im Service	23
3.2 Auslesen der 10 Features	24
3.3 On-Device-Inferenz mit TFLite	25
3.4 Robustheit – die App muss überleben	26
3.5 Schema-Drift im Logger	27
3.6 Python-Pipeline – Beispiel Segment-Erkennung	28
3.7 Trainings-Verlauf	28
3.8 Code-Umfang und Eigenleistung	29
3.9 Reproduzierbarkeit der Pipeline	30
4 Abschließender Erfahrungsbericht	32
4.1 Wöchentliche Einträge	32

4.2	Schlüssel-Entscheidungen, Fehler und Trade-offs	35
4.3	Gesamt-Aufwand	37
4.4	KI-Nutzung	37
4.5	Abschluss-Reflexion	39
4.6	Grenzen der Arbeit und Ausblick	40
Literatur		42

Abbildungsverzeichnis

1	Vereinfachter Datenfluss zwischen den Systemkomponenten	6
2	Detaillierte Gesamtarchitektur des Battery-Predictor-Systems	7
3	Onboarding-Slides 1 und 4. Slides 2 und 3 dazwischen erklären Sensoren und PC-Training.	10
4	Live-Tab im Zustand „Lädt“: das Label oben („STATUS“) wechselt dynamisch, damit nicht „AKKU NOCH Lädt“ entsteht. Confidence-Pille ist in diesem Zustand bewusst ausgeblendet (Vorhersage pausiert).	11
5	Algorithmus-Tab, obere zwei Visualisierungen.	12
6	Algorithmus-Tab, untere Sektionen: Live-Sparkline und USP-Pillen.	13
7	Genauigkeits-Tab: für den letzten beobachteten Discharge-Lauf wird der MAE der drei Methoden (eigenes TinyML, Google API, Linear-Baseline) verglichen. Hier auf dem Xiaomi: alle drei „>10 h daneben“ – bewusst ehrliche Anzeige eines Falls, in dem das Modell nicht überzeugt. Die Statistik aus dem Paper trennt sauberer.	14
8	Settings-Activity. Hinweis: der Button „Onboarding noch einmal ansehen“ ist exakt für den Fall gedacht, dass der Prüfer das Onboarding versehentlich weggeklickt hat.	15
9	Train-Own-Model-Activity, aufgerufen über den Settings-Button „Modell selbst trainieren“.	16
10	Trainings- und Validation-MAE des Conv1D-Modells. Auf dem Multi-Device-Datensatz erreicht das Modell nach 21 Epochen ein Validation-MAE von ca. 5 h. Da EarlyStopping mit <code>restore_best_weights=True</code> konfiguriert ist, wird das beste Modell zurückgegeben.	29
11	Die vier Wendepunkte der Projekt-Story. Orange: bewusste Entscheidung, Rot: vermiedener Irrtum. Gestrichelte Linien deuten an, dass sich der Stats-Aufwand in Woche 2 erst eine Woche später beim Leakage-Schock auszahlt.	35

Tabellenverzeichnis

1	Projekt auf einen Blick	4
2	Module der Android-App	8
3	Felder eines CSV-Eintrags	19
4	Im Manifest deklarierte Permissions	21
5	Berechtigungen mit Nutzer-Aktion	21
6	Code-Umfang nach Bereich	29
7	Schlüssel-Entscheidungen und verworfene Alternativen	36
8	Zeitaufwand nach Woche	37

Quellcode-Verzeichnis

1	Mess-Schleife im DataCollectorService	23
2	CPU-Last als Frequenz-Proxy	24
3	App-Kategorisierung (Auszug)	24
4	Hotspot-Status via Reflection	25
5	Inferenz mit TFLite	25
6	Auto-Restart bei onTaskRemoved	27
7	Schema-Drift im Logger abfangen	27
8	Discharge-Segmente finden	28
9	Pipeline reproduzierbar starten	30

0 Ausgangssituation und Zielsetzung

0.1 Alleinstellungsmerkmal

Eine eigene Akku-Vorhersage durch ein 14,4 KB großes TinyML-Modell, das vollständig offline auf jedem Android-Gerät läuft – auch dort, wo Googles eigene Anzeige fehlt.

Das ist das eine Feature, um das sich die gesamte Arbeit dreht. Alle weiteren Entscheidungen – von der Sensorauswahl über die Modell-Architektur bis zum UI-Refactor – ordnen sich diesem Hauptfeature unter.

Das mitgelieferte Conv1D-Netz ist mit 14,4 KB kleiner als eine WhatsApp-Sprachnachricht, läuft INT8-quantisiert auf jeder Android-Version ohne spürbaren Energie- oder Speicher-Overhead und hält sämtliche Sensordaten lokal – es gibt keinen Server-Roundtrip. Daten werden in der App gesammelt, am PC trainiert und das fertige TFLite-Modell wieder zurück in die App deployt; die App ist damit nicht nur Konsument eines vortrainierten Modells, sondern Teil einer schließbaren Trainings-Schleife, die geräte- und nutzerspezifisch nachjustiert werden kann.

Auf älteren Geräten ist die App *überhaupt* die einzige verfügbare ML-Vorhersage: erst mit Android 12 (Oktober 2021) hat Google eine ML-basierte Akku-Schätzung ins System eingebaut – alle Android-Versionen davor (also über zehn Jahre Android-Geräte) bieten nur die einfache lineare Stundenanzeige aus dem Jahr 2014. Auf einem Smartphone, das nie ein Android 12-Update bekommen hat, ist diese App damit die einzige Möglichkeit, eine neuronale Restlaufzeit-Vorhersage zu bekommen.

0.2 Problemstellung

Smartphone-Akkuanzeigen funktionieren auf den meisten Geräten seit Android 5 (2014) nach einem einfachen Prinzip: man rechnet den durchschnittlichen Verbrauch der letzten Stunden linear in die Zukunft. Wer schon einmal mit 30 % Akku „noch 4 Stunden“ im Display hatte und dann nach einem YouTube-Video plötzlich bei 15 % mit „noch 1 Stunde“ stand, kennt das Problem.

Akku-Verbrauch ist in Wahrheit nichtlinear und vom Kontext getrieben: Streaming verbraucht ein Vielfaches von Lesen, Navigation in der Kälte mehr als bei Wärme, eine schwache Mobilfunk-Verbindung mehr als WLAN, ein heißes Spiel mehr als der Sperrbildschirm. Ein linearer Durchschnitt der letzten Minuten kann diese Dynamik prinzipbedingt nicht abbilden – dafür braucht es ein Modell, das die Beziehung zwischen Sensoren und Verbrauch tatsächlich *gelernt* hat.

Genau das ist auf den meisten Smartphones bis heute nicht vorhanden: erst mit Android 12 (Oktober 2021) hat Google überhaupt ein eigenes ML-Modell ins System eingebaut (`PowerManager.getBatteryDischargePrediction()`) [2]. Für alle Android-Versionen davor – und damit für einen großen Teil der weltweit aktiven Geräte – gibt es weiterhin nur die einfache lineare Schätzung aus dem Jahr 2014. Eine echte, an das eigene Nutzungsverhalten angepasste KI-Vorhersage existiert dort schlicht nicht.

0.3 Konkrete Anwendungsszenarien

Damit klar wird, für wen das überhaupt relevant ist – vier Alltagssituationen, in denen eine ehrliche Restlaufzeit messbar hilft. Allen gemeinsam: die nächste Steckdose ist eben nicht in fünf Minuten erreichbar.

- **Beim Wandern oder auf Mehrtagestouren.** Eine Restlaufzeit, der man trauen kann, entscheidet, ob man unterwegs noch die Karte öffnen kann oder lieber den Flugmodus aktiviert. Genau das, was ein an deinem Nutzungsverhalten trainiertes TinyML-Modell deutlich besser hinbekommt als ein einfacher linearer Durchschnitt.
- **Auf Reisen ohne sichere Ladequelle.** Im Zug, im Flugzeug, im Hotelzimmer ohne freie Steckdose. Statt nervös alle zehn Minuten zu prüfen, gibt eine verlässliche Schätzung Planungssicherheit (z. B. ob man Filme bis zur Ankunft schauen kann).
- **Im Pendel-Alltag.** „Reicht der Akku noch für den Heimweg, oder muss ich im Büro noch laden?“ Eine ehrliche Restlaufzeit am Vormittag erlaubt die richtige Entscheidung am Mittag.
- **Beim aktiven Tagesplanen.** Wer das Handy für Navigation, Musik und Foto braucht (Konzert, Festival, Stadtführung), nutzt das Gerät wesentlich aggressiver als an einem normalen Büroarbeitstag. Ein Modell, das die zehn Sensor-Features in genau diesem Verhaltensmuster gelernt hat, ist hier deutlich nützlicher als ein generischer Durchschnitt.

Allen Szenarien gemeinsam: weniger Stress beim Ladesuchen, kein hektisches Powerbank-Kramen, kein „hätte ich das früher gewusst“. Genau diese vier Situationen sind als Slide 1 im Onboarding der App hinterlegt – so sieht der Nutzer beim ersten Öffnen sofort, wofür sie gemacht ist.

0.4 Mehrwert aus Nutzer- und Marktsicht

Aus Nutzer- und Marktsicht steht die App auf fünf Punkten:

- **Eigene Architektur, nicht nur eigene UI.** Kern ist ein selbst trainiertes TinyML-Modell, das auf dem Gerät läuft – nicht ein hübscheres Frontend für eine vorhandene

API. Das unterscheidet die App von jeder anderen Akku-Anzeige.

- **Auf das eigene Gerät zuschneidbar.** Das mitgelieferte Modell läuft auf jedem Android-Gerät als Basis; jeder Nutzer kann es darüber hinaus auf den eigenen gesammelten Daten nachtrainieren und so auf die Eigenheiten seines Geräts und seines Nutzungsverhaltens zuschneiden. Eine generische System-Schätzung oder Cloud-API hat diese Möglichkeit nicht – dort bekommt jedes Gerät dieselbe Vorhersage-Logik.
- **Konkreter Nutzen.** Weniger „reicht der Akku noch?“-Momente, weniger spontaner Powerbank-Kauf, bessere Planung von Zeitfenstern ohne Steckdose. Die vier Szenarien aus Abschnitt 0.3 machen das greifbar.
- **Keine laufenden Kosten.** Inferenz läuft komplett auf dem Endgerät. Eine Verteilung an 1.000 oder 10.000 Nutzer kostet dasselbe wie an einen einzigen: nichts. Kein Cloud-Compute, kein API-Call pro Vorhersage, keine variable Infrastruktur in der Kostenrechnung.
- **Niedrige Einstiegshürde.** Kein Account, kein Abo, keine Cloud-Anmeldung. Onboarding in unter 90 Sekunden, direkt danach die erste Vorhersage.

0.5 Zielsetzung

Ziel ist eine Android-App, die

- alle 30 Sekunden 10 Sensordaten-Features sammelt (Akku, Helligkeit, App-Kategorie, CPU, Temperatur usw.),
- ein eigenes TinyML-Modell on-device einsetzt,
- parallel die Google-API liest und vergleicht,
- alles in eine CSV schreibt, die am PC ausgewertet wird.

Funktional sollte am Ende stehen:

1. Eine Android-App, die im Hintergrund Daten sammelt und überlebt (Foreground-Service, Auto-Restart nach Boot oder Wegwischen).
2. Ein quantisiertes TFLite-Modell (< 15 KB), das in der App on-device läuft und eine Vorhersage in der Notification anzeigt.
3. Eine Trainings-Pipeline am PC, die die exportierte CSV verarbeitet, das Modell trainiert und es zurück in die App deployt.

4. Eine Auswertung, die zeigt, wie gut das eigene Modell gegen die Google-API und einfache Baselines steht.

Dieses Portfolio dokumentiert den *Bau-Aufwand*, die Architektur und die Programmier-Eigenleistung. Die *wissenschaftliche* Auswertung des Multi-Device-Vergleichs ist Gegenstand der separaten Paper-Einreichung (`paper/main.pdf` im selben Repo) und nicht Inhalt dieses Dokuments.

0.6 Rahmenbedingungen

- **Eigenes Gerät:** Xiaomi 2107113SG (Redmi Note 10 Pro), Android 12, 8 GB RAM.
- **Zusätzliche Test-Geräte** (Familie): Google Pixel 7 Pro, Pixel 8 Pro, Pixel 9 Pro XL.
- **Entwicklungsumgebung:** Android Studio (Kotlin), Python 3.13 mit TensorFlow 2.21 am Windows-Notebook.
- **Bearbeitungszeitraum:** 25. März bis 11. Mai 2026 (sieben Wochen).

0.7 Eckdaten des Projekts

Das Projekt umfasst drei Schichten – die Android-App selbst, eine Python-Pipeline am PC für Training und Auswertung, sowie das zurück-deployte TFLite-Modell für die On-Device-Vorhersage. Die folgenden Kennzahlen geben einen ersten Eindruck vom Umfang; Details zu jeder Schicht finden sich in den entsprechenden Hauptkapiteln.

Tabelle 1: Projekt auf einen Blick

Kennzahl	Wert
Bearbeitungszeit	7 Wochen, ≈ 135 h Eigenleistung
Code-Umfang gesamt	≈ 5.600 Zeilen (Kotlin + Python + XML)
Android-App	18 Kotlin-Module (App-Lifecycle, Service, Daten, UI-Tabs + Onboarding + Settings, Predictor, Confidence, Accuracy, Algorithm-View + drei Custom-Visualisierungen inkl. Modell-Architektur)
Python-Pipeline	25 Module in zwei Schichten (<code>methods/</code> , <code>evaluation/</code>)
Test-Geräte	4 (Xiaomi 2107113SG + Pixel 7 / 8 / 9 Pro)
Mitwirkende	3 Familienmitglieder für APK-Rollout und Pixel-Datensammlung
Feld-Sammlung	45 Tage Dauerbetrieb, 180 Service-Sessions
Datensatz	66.001 Messpunkte $\times 10$ Features
TFLite-Modell	14.4 KB INT8-quantisiert, $\sim 5 \mu s$ pro Inferenz
App-UI	Vier-Slide-Onboarding mit Use-Cases + Modell-Größen-Vergleich, drei Tabs (Live / Algorithmus / Genauigkeit), Settings-Screen mit Onboarding-Replay

1 Architekturmodell

1.1 Was passiert im Inneren – in einfachen Worten

Bevor wir technisch werden: hier eine bewusst nicht-technische Beschreibung dessen, was die App tatsächlich tut. Wer Akkuanzeigen kennt, hat in 60 Sekunden den ganzen Mechanismus verstanden:

1. **Daten sammeln (Beobachten):** Alle 30 Sekunden schreibt die App zehn Werte vom Handy auf – zum Beispiel den aktuellen Akkustand, die Bildschirm-Helligkeit, ob gerade YouTube läuft, die Temperatur des Geräts und einige mehr. Das ist nichts anderes, als wenn man stündlich Tagebuch über sein eigenes Verhalten schreiben würde. Über mehrere Wochen entsteht so eine Tabelle mit über 66.000 Zeilen.
2. **Muster lernen (Training auf dem PC):** Diese Tabelle wird auf den Laptop kopiert und einem kleinen Programm (dem „Modell“) gezeigt. Das Programm lernt Zusammenhänge wie: *Wenn die Helligkeit hoch ist, YouTube läuft und die Temperatur 38°C beträgt, dann hält der Akku noch ungefähr 2 Stunden.* Es lernt diese Zusammenhänge nicht aus Büchern, sondern indem es die Tabelle viele Male durchrechnet und seine Vorhersagen mit dem tatsächlichen Akku-Verlauf abgleicht.
3. **Im Alltag vorhersagen (Modell zurück im Handy):** Das fertig gelernte Modell ist nur 14,4KB groß – kleiner als ein WhatsApp-Sprachmemo. Es wandert zurück in die App, läuft dort ohne Internetverbindung und sagt im Hintergrund alle 30 Sekunden eine neue Restlaufzeit voraus. Zum Vergleich zeigt die App gleichzeitig Googles eigene Vorhersage und einen ganz simplen Wert nach dem Motto „so wie es bisher lief, läuft es weiter“.

Warum genau diese Aufteilung? Die drei Schritte sind absichtlich auf zwei Geräte aufgeteilt (Handy für Schritte 1 und 3, PC für Schritt 2). Das hat zwei Gründe:

- Auf dem Handy ist das Trainieren des Modells zu rechenintensiv – es würde den Akku stark belasten und Stunden dauern. Auf dem PC ist es in wenigen Minuten erledigt.
- Das fertige Modell ist dafür so klein, dass es im Handy problemlos im Hintergrund läuft, ohne dass der Nutzer etwas davon merkt. Diese Asymmetrie – *trainieren ist teuer, vorhersagen ist billig* – ist typisch für ML-Systeme und der Grund, warum es überhaupt sinnvoll ist, ein Modell auf einem Gerät einzusetzen.

Verortung im Kurs „Mobile und Ubiquitäre Anwendungen“. Die Arbeit berührt drei Themen der Vorlesung direkt:

- *Sensoren und Datenerhebung am mobilen Endgerät* – die App liest zehn unterschied-

liche Android-Signale aus und zeigt, welche Permissions und APIs dafür nötig sind.

- *Background-Services und Lebenszyklus* – ein Foreground-Service hält die Datensammlung am Laufen, auch wenn die App vom Nutzer aus dem Speicher gewischt wird. Hier wird das im Kurs behandelte Service-Konzept ganz konkret durchgespielt.
- *Anwendungsbezogene Auswertung mobiler Daten* – statt die Daten zu sammeln und in der Cloud zu verarbeiten, bleibt alles auf dem Gerät. Das ist eine bewusste Design-Entscheidung zugunsten des Nutzers, nicht der Bequemlichkeit des Entwicklers.

Wer aus dem Portfolio nur eine Sache mitnehmen soll, dann diese: *Das Projekt löst eine Standard-App-Funktion (Akkuanzeige) mit einem 14,4 KB großen TinyML-Modell, das vollständig auf dem Gerät läuft und auf den eigenen Daten des Nutzers nachjustiert werden kann – und legt dabei jeden Zwischenschritt offen.*

1.2 Gesamtarchitektur im Überblick

Das System gliedert sich in zwei klar getrennte Hälften, die durch CSV-Dateien und das .tflite-Modell als Schnittstellen kommunizieren:

1. **Datenerhebung (Android-App)** – alle 30 Sekunden ein Datenpunkt mit 10 Features, gespeichert in CSV.
2. **Modelltraining (Python-Pipeline)** – liest die konsolidierte CSV, trainiert Conv1D, konvertiert nach TFLite.
3. **On-Device-Inferenz (Android-App)** – das TFLite-Modell läuft direkt im Phone und liefert die Vorhersage in der Notification.

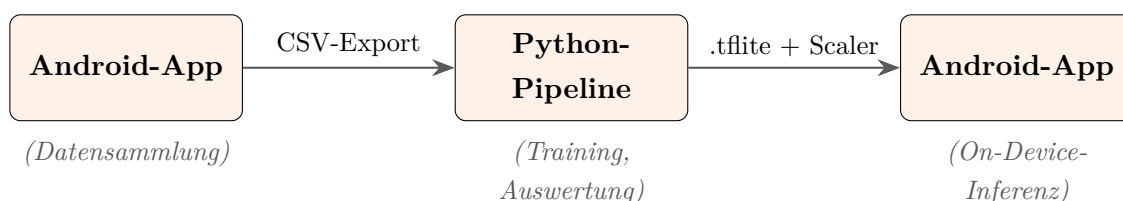


Abbildung 1: Vereinfachter Datenfluss zwischen den Systemkomponenten

Die vollständige Detail-Architektur mit allen Unterkomponenten, internen Verbindungen und Schnittstellen zeigt Abbildung 2 auf der folgenden Seite.

Gesamtarchitektur: Akkuvorhersage mit TinyML

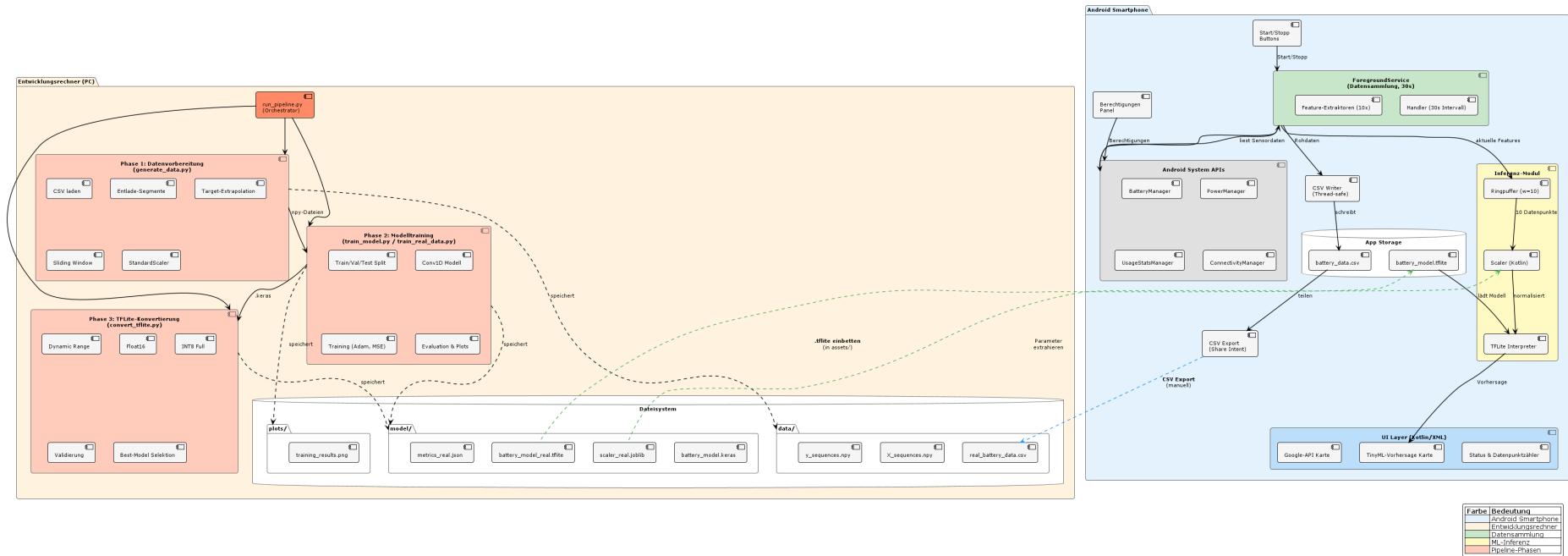


Abbildung 2: Detaillierte Gesamtarchitektur des Battery-Predictor-Systems

1.3 Android-App im Detail

Die App ist in achtzehn Kotlin-Dateien gegliedert, sauber getrennt in App-Lifecycle, Datenerhebungs-Service, Modell-Inferenz und UI-Bausteine:

Tabelle 2: Module der Android-App

Datei	Aufgabe
<i>Datenerhebung und Modell</i>	
<code>DataCollectorService.kt</code>	Foreground-Service, sammelt alle 30 Sekunden Daten und sendet Broadcasts an die UI
<code>BatteryDataPoint.kt</code>	Datenklasse für einen Messpunkt mit CSV-Serialisierung
<code>BatteryDataLogger.kt</code>	Schreibt CSV in <code>filesDir</code> , behandelt Schema-Drift und Legacy-Migration
<code>BatteryPredictor.kt</code>	TFLite-Interpreter mit StandardScaler-Normalisierung
<code>BootReceiver.kt</code>	Startet den Service nach Geräte-Neustart
<i>App-Lifecycle und Onboarding</i>	
<code>MainActivity.kt</code>	Tab-Container mit Bottom-Navigation, Settings-Icon, Onboarding-Weiche
<code>OnboardingActivity.kt</code>	Drei-Slide-Erstkontakt mit Versprechen und Modell-Größen-Vergleich (14,4 KB)
<code>OnboardingPagerAdapter.kt</code>	ViewPager2-Adapter für die Onboarding-Slides
<i>UI-Tabs (Hauptbühne)</i>	
<code>LiveFragment.kt</code>	Hauptvorhersage gross, Confidence-Pille, Live-Anzeige der 10 Sensor-Werte als animierte Karten
<code>AlgorithmFragment.kt</code>	Erklärt in vier Schritten + USP-Block, wie aus Sensoren die Vorhersage wird; bindet die zwei Visualisierungs-Views ein
<code>DataflowView.kt</code>	Custom Canvas-View: animierter Datenfluss von 10 pulsierenden Sensoren durch ein leuchtendes TinyML-Modell zur Vorhersage
<code>BatteryHistoryChart.kt</code>	Custom Canvas-View: Sparkline der letzten 3 Stunden Akkuverlauf + gestrichelte Vorhersage-Extension für die nächste Stunde
<code>ConvLayerView.kt</code>	Custom Canvas-View: visualisiert die Conv1D-Architektur selbst – Input-Matrix, 16 Feature-Maps, Pooling, Dense-Output mit pulsierender Forward-Pass-Animation

Fortsetzung auf nächster Seite

Fortsetzung Tabelle 2

Datei	Aufgabe
<code>AccuracyFragment.kt</code>	Genauigkeits-Tab mit 3-Wege-Score (TinyML / Google / Linear)
<code>AccuracyComputer.kt</code>	Liest die CSV-Daten und berechnet den MAE der drei Methoden im letzten Discharge-Lauf
<i>Infrastruktur-UI</i>	
<code>SettingsActivity.kt</code>	Service-Status, Permissions, CSV-Export, Modell-Info – bewusst hinter dem Zahnrad-Icon
<code>ConfidenceBadge.kt</code>	Wiederverwendbare Pille mit Punkt und Label (grün / gelb / rot)
<code>ConfidenceLevel.kt</code>	Enum + Heuristik (Buffer-Größe + Akkustand)

1.4 Die App in Aktion – Bedienungsanleitung mit Screenshots

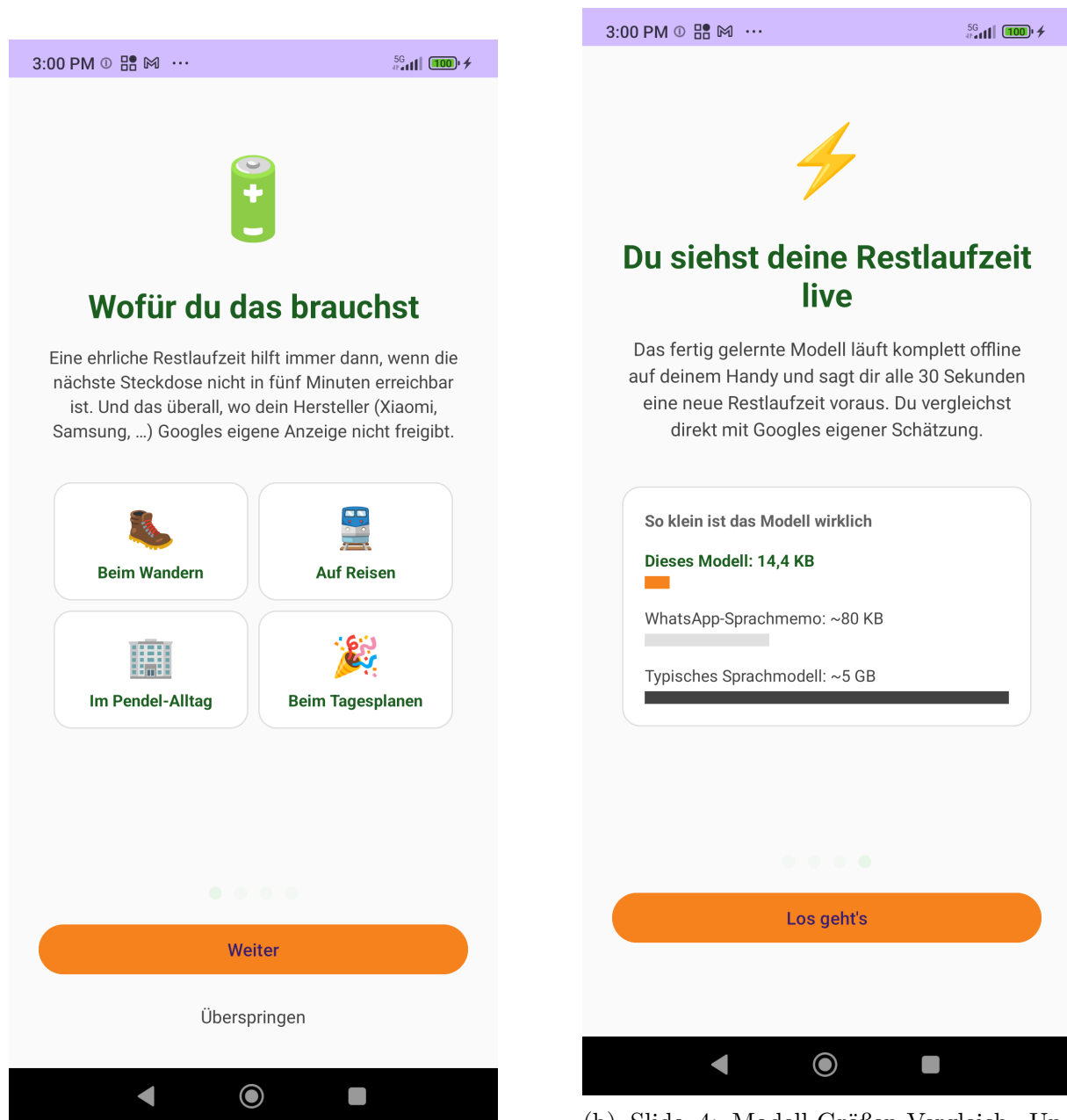
Diese Sektion gibt eine vollständige Bedienungsanleitung der App im finalen Zustand nach dem UI-Refactor in Woche 7. Sie ergänzt die textuelle Modul-Beschreibung in Tabelle 2 um Screenshots aller relevanten Bildschirme.

Finale UI nach UI-Refactor

Hinweis vorab: in der ersten Projekt-Hälfte (Wochen 1–6) bestand die App aus einer einzigen Activity mit Start-/Stopp-Buttons, einer zentralen Live-Vorhersage und einem Datenpunkt-Zähler. Das war ausreichend für die Datensammlung, aber für die Abgabe zu rau – deshalb der UI-Refactor in Woche 7, dessen Ergebnis im Folgenden gezeigt wird.

Im Zuge des Portfolio-Abschlusses wurde die UI vollständig überarbeitet, mit klarem Fokus auf das eine USP-Feature (eigene Vorhersage transparent sichtbar machen). Der Refactor folgt explizit den etablierten Mobile-Onboarding- und Explainable-AI- Best-Practices: vier Slides statt fünf (App-Onboarding-Studien zeigen einen deutlichen Abfall der Completion-Rate ab dem fünften Schritt), Confidence als visueller Indikator statt nackter Zahl, das eigentliche Feature direkt auf der Hauptbühne und Infrastruktur-Knöpfe in einem separaten Settings-Screen.

Welcome-Bildschirm: Onboarding (vier Slides). Beim allerersten Öffnen wird automatisch das vierteilige Onboarding gezeigt. Slide 1 verortet den Nutzen anhand vier Alltagssituationen, Slide 2–3 erklären das „Sammeln → Lernen → Vorhersagen“-Prinzip in einfachen Worten, Slide 4 zeigt den Modell-Größen-Vergleich als Wow-Moment.



(a) Slide 1: Vier Anwendungsszenarien als 2×2-Grid (Wandern, Reisen, Pendel-Alltag, Tagesplanen).

(b) Slide 4: Modell-Größen-Vergleich. Unser Modell (14,4 KB) in Orange, daneben WhatsApp-Sprachmemo und ein typisches Sprachmodell.

Abbildung 3: Onboarding-Slides 1 und 4. Slides 2 und 3 dazwischen erklären Sensoren und PC-Training.

Tab „Live“: das Hauptfeature. Direkt nach dem Onboarding (oder bei jedem späteren Öffnen) landet der Nutzer im Live-Tab. Oben groß die Restlaufzeit (oder ein expliziter Status „Lädt“ / „Voll“ mit passender Sub-Erklärung), darunter alle zehn Live-Sensoren als animierte Karten – die Hauptbühne der App.

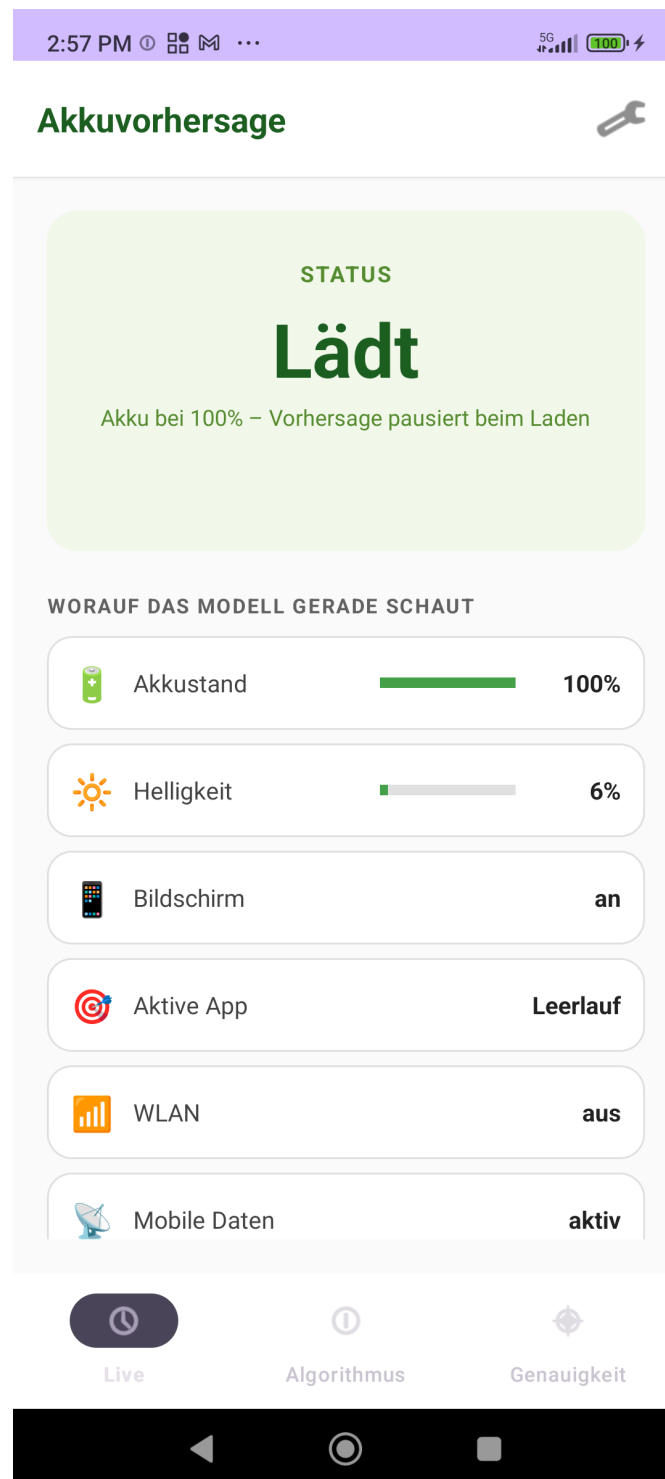


Abbildung 4: Live-Tab im Zustand „Lädt“: das Label oben („STATUS“) wechselt dynamisch, damit nicht „AKKU NOCH Lädt“ entsteht. Confidence-Pille ist in diesem Zustand bewusst ausgeblendet (Vorhersage pausiert).

Tab „Algorithmus“: die drei Visualisierungen. Der mittlere Tab erklärt, wie der Algorithmus tatsächlich funktioniert. Drei eigene Custom-Canvas-Views, die alle live laufen.

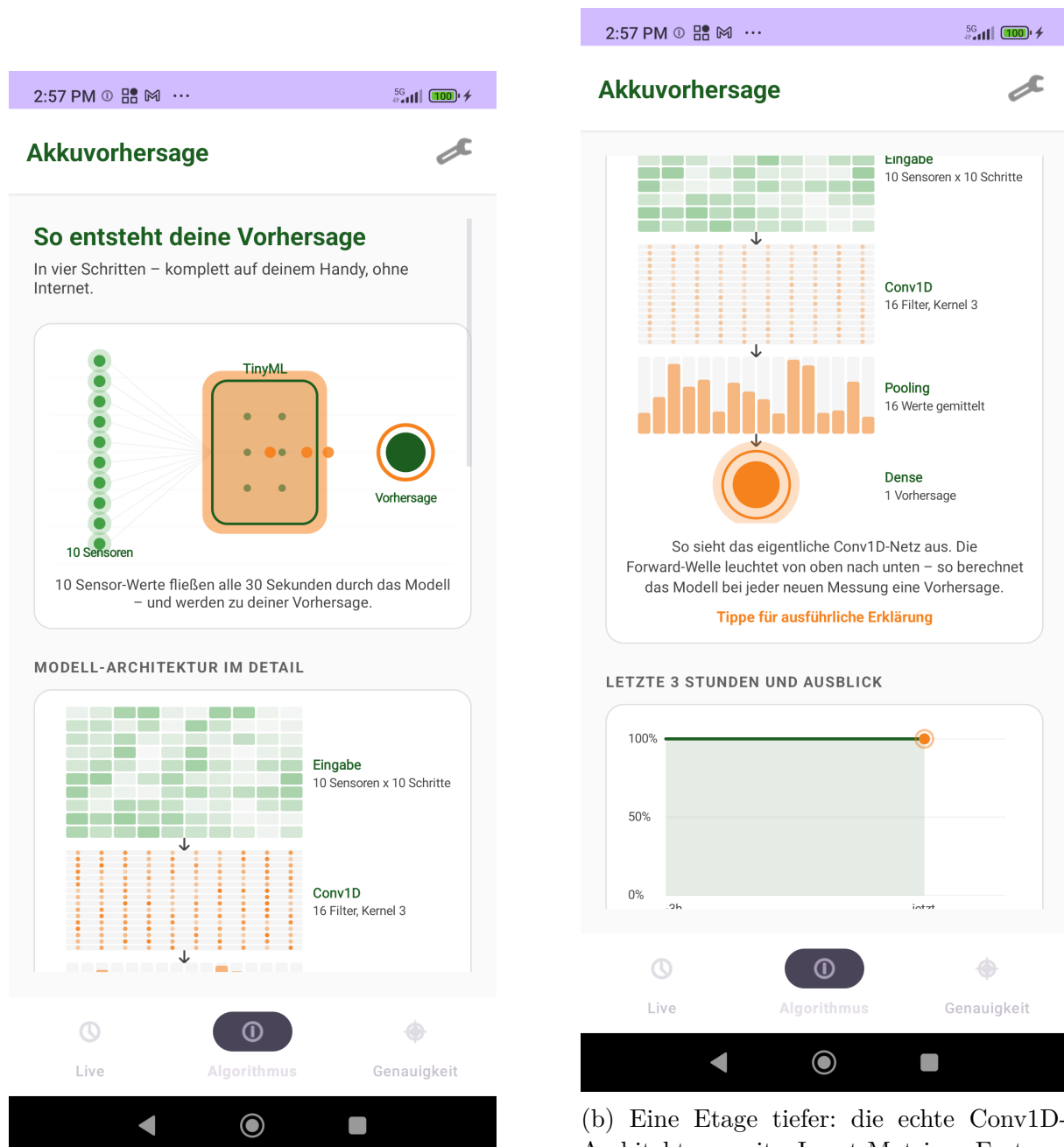
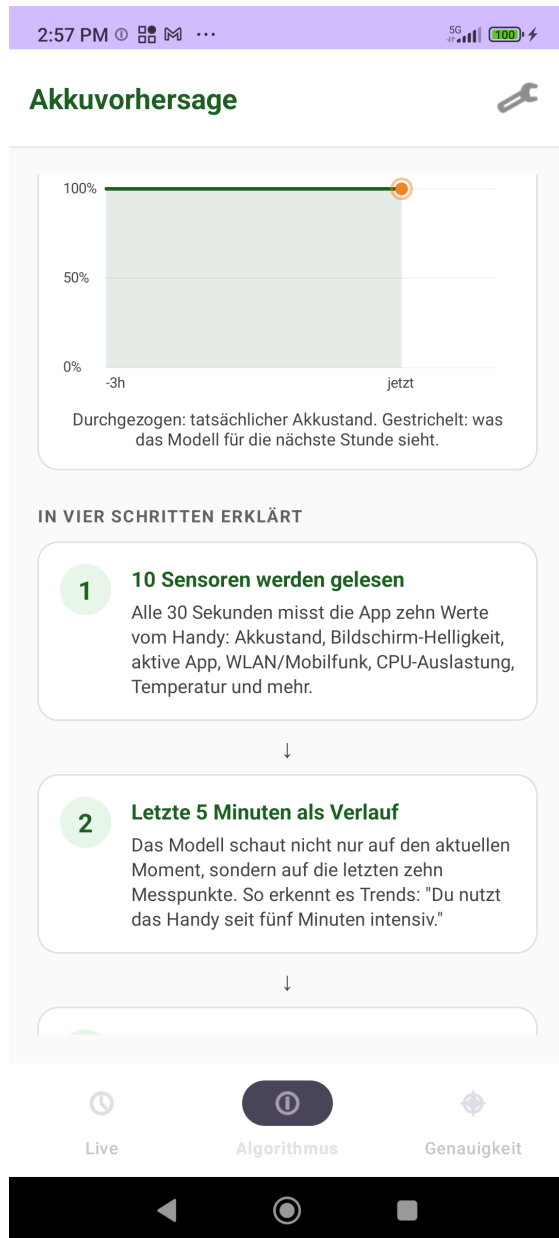


Abbildung 5: Algorithmus-Tab, obere zwei Visualisierungen.



(a) Live-Sparkline der letzten drei Stunden Akkuverlauf plus gestrichelte Vorhersage-Extension. Hier (Akku zu 100 %) nur als Linie sichtbar, weil keine Discharge-Dynamik vorhanden.



(b) Ganz unten der USP-Block „Was es besonders macht“: offline, 14,4 KB, universal.

Abbildung 6: Algorithmus-Tab, untere Sektionen: Live-Sparkline und USP-Pillen.

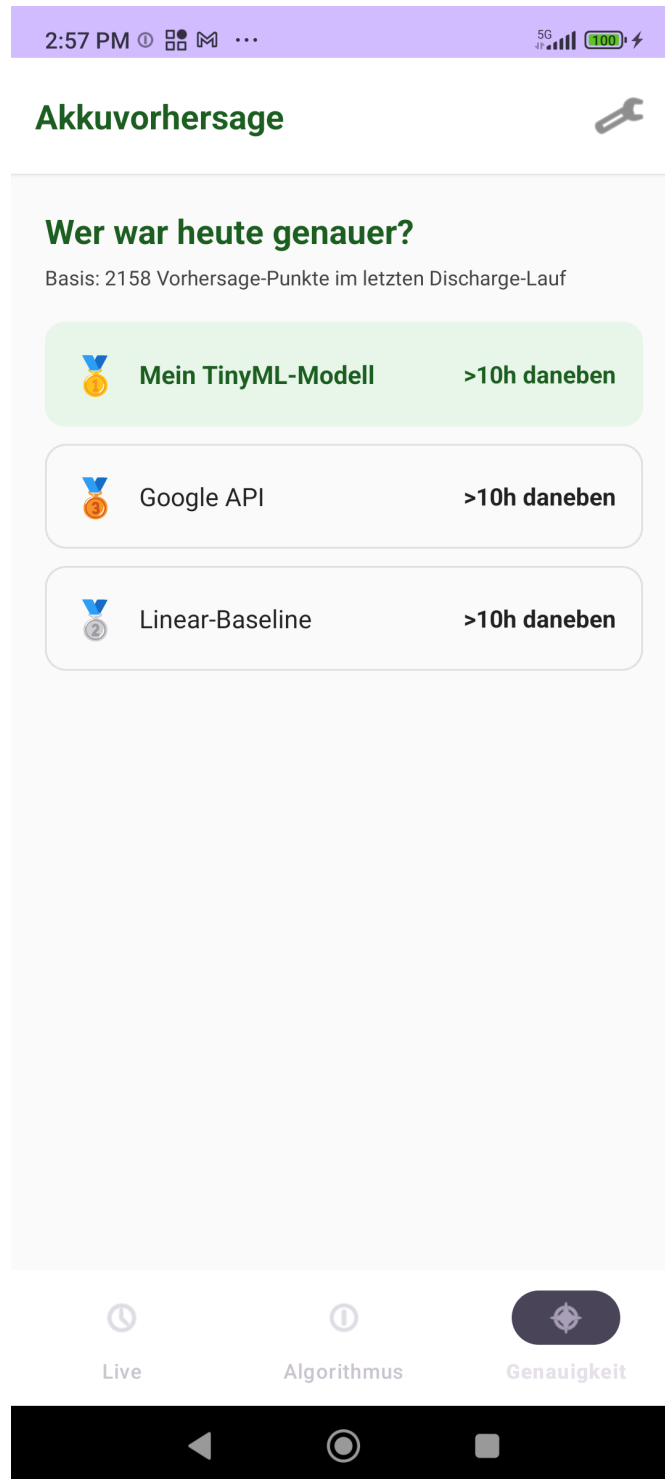
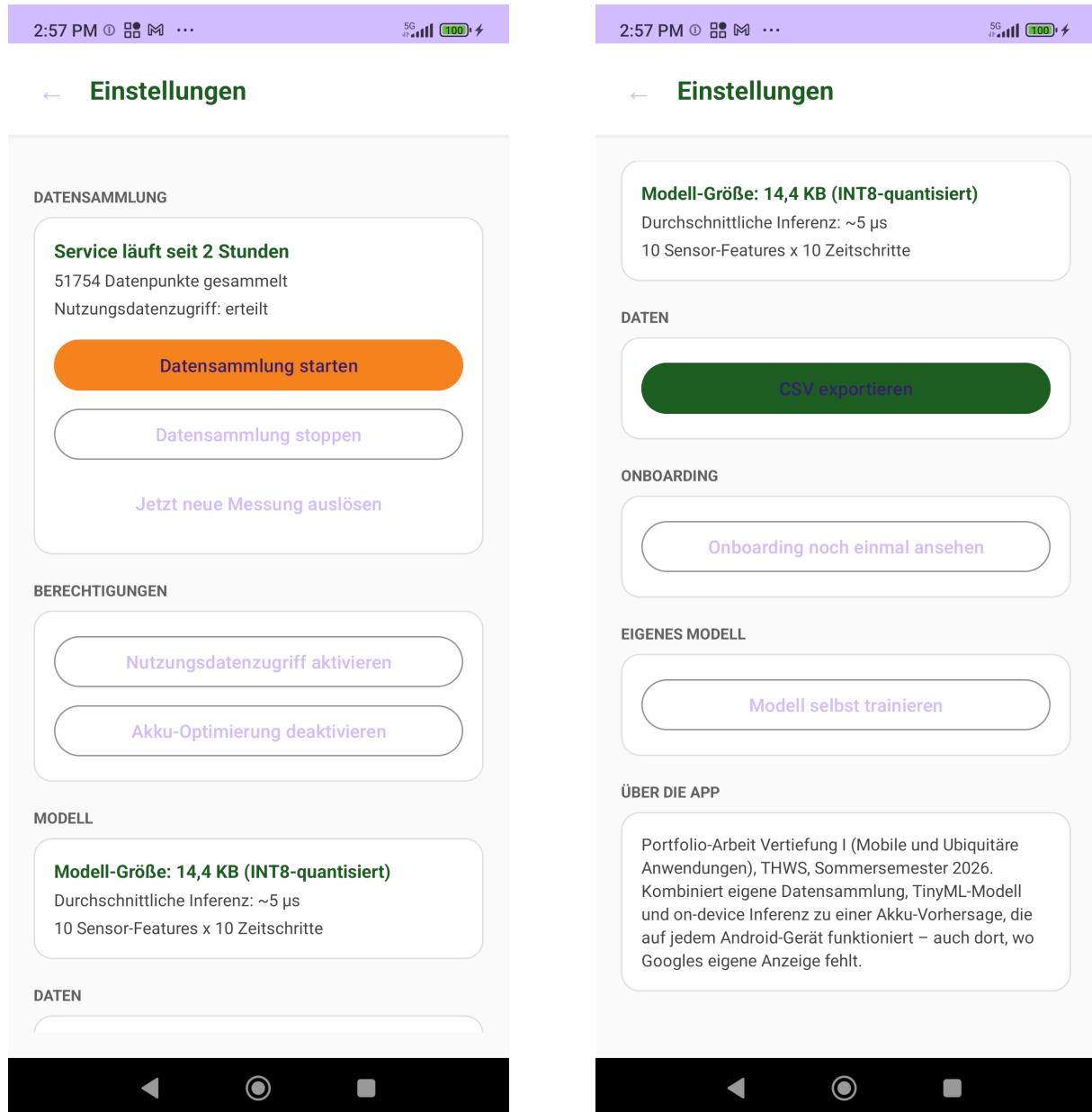


Abbildung 7: Genauigkeits-Tab: für den letzten beobachteten Discharge-Lauf wird der MAE der drei Methoden (eigenes TinyML, Google API, Linear-Baseline) verglichen. Hier auf dem Xiaomi: alle drei „>10 h daneben“ – bewusst ehrliche Anzeige eines Falls, in dem das Modell nicht überzeugt. Die Statistik aus dem Paper trennt sauberer.

Tab „Genauigkeit“: der ehrliche Vergleich.

Settings (Zahnrad-Icon oben rechts). Hier liegt alles, was nicht zur Hauptbühne gehört: Service- Steuerung, Permissions, Modell-Info, CSV-Export – und genau dort findet der Nutzer auch den Button, um das Onboarding noch einmal anzusehen oder ein eigenes Modell zu trainieren.

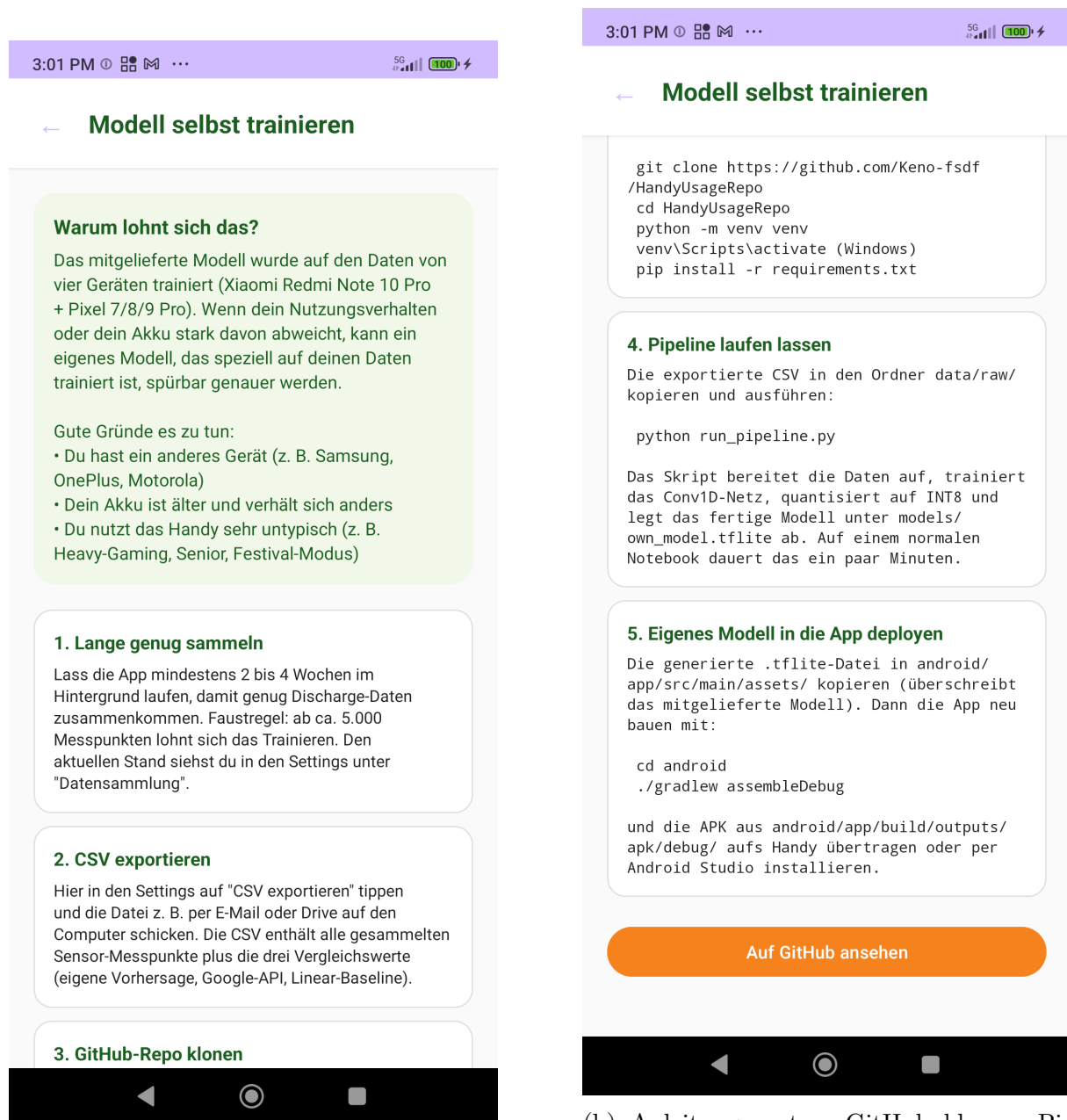


(a) Settings, oberer Teil: Service-Status mit Laufzeit und Datenpunkt-Counter, Start-/Stopp-Buttons, Berechtigungs-Buttons und Modell-Info.

(b) Settings, unterer Teil: CSV-Export, der wichtige Button „Onboarding noch einmal ansehen“ und der Einstieg in den Trainings-Guide „Modell selbst trainieren“.

Abbildung 8: Settings-Activity. Hinweis: der Button „Onboarding noch einmal ansehen“ ist exakt für den Fall gedacht, dass der Prüfer das Onboarding versehentlich weggeklickt hat.

Eigenes Modell trainieren (aus Settings erreichbar). Wer das mitgelieferte Modell durch eines ersetzen möchte, das auf den eigenen 45+ Tagen Daten trainiert ist, findet hier eine Anleitung in fünf Schritten inklusive Repo-Link.



(a) Anleitung, oben: Warum-Block plus die ersten zwei Schritte (lange sammeln, CSV exportieren).

(b) Anleitung, unten: GitHub klonen, Pipeline laufen lassen, .tflite zurück in android/app/assets/ deployen. Großer „Auf GitHub ansehen“-Button.

Abbildung 9: Train-Own-Model-Activity, aufgerufen über den Settings-Button „Modell selbst trainieren“.

Detail-Verweise auf die finale UI. Die folgenden Punkte fassen die zentralen Design-Entscheidungen des Refactors zusammen – sie spiegeln, was die Screenshots oben zeigen:

- *Vier-Slide-Onboarding* beim ersten Start. Erklärt in einfachen Worten, was die App tut, und zeigt im vierten Slide einen Größen-Vergleich „Unser Modell: 14,4 KB – WhatsApp-Sprachmemo: ~ 80 KB – typisches Sprachmodell: ~ 5 GB“ als Wow-Element.
- *Tab „Live“*. Die Hauptbühne der App. Vorhersage prominent oben, direkt darunter eine *Confidence-Pille* (grün / gelb / rot) und ein Live-Block mit allen zehn Sensoren als animierte Karten – der Nutzer „sieht dem Modell beim Denken zu“.
- *Tab „Algorithmus“*. Eigener Tab für die Erklärung des USP-Features. Ganz oben – gleich beim Öffnen sichtbar – drei Pillen mit den Alleinstellungs-Argumenten (**offline**, **14,4 KB**, **jedes Android**), darunter drei aufeinander aufbauende Visualisierungen:
 - *Datenfluss* – zehn pulsierende Sensoren links, leuchtende TinyML-Box in der Mitte, fließende Partikel hinüber zur Vorhersage rechts (high-level Überblick).
 - *Modell-Architektur* – direkt darunter die eigentlichen Conv1D-Schichten als Detail-Ansicht: Input-Matrix 10×10 , 16 Feature-Maps, 16 gemittelte Pool-Werte, ein Dense-Output. Eine sequentielle Forward-Pass-Animation lässt eine „Welle“ von oben nach unten durch die Schichten laufen, damit nachvollziehbar wird, wie das Modell intern rechnet.
 - *Live-Sparkline* der letzten drei Stunden Akkuverlauf mit gestrichelter Vorhersage-Extension für die nächste Stunde (praktische Anwendung des Modells in Echtzeit).

Danach die textuellen vier nummerierten Schritte (Sensoren $\rightarrow$ Sliding-Window $\rightarrow$ TinyML $\rightarrow$ Vorhersage) und ein USP-Block am Ende (komplett offline, 14,4 KB, jedes Android). Macht den Algorithmus selbst zu einem sichtbaren, animierten App-Element statt nur Black-Box-Output.

- *Tab „Genauigkeit“*. Zeigt einen 3-Wege-Score „Wer war heute genauer?“ mit Medail-
len (eigenes Modell vs. Google-API vs. Linear-Baseline), berechnet über den letzten beobachteten Discharge-Lauf.
- *Settings hinter dem Zahnrad*. Alle Infrastruktur- Bedienelemente (Service-Status mit Laufzeit, Permissions-Check, Start/Stopp, CSV-Export, Modell-Info-Karte) sind bewusst aus der Hauptbühne genommen, damit der Nutzer beim Öffnen der App genau *ein* Feature sieht und nicht ein Feature-Buffer.
- *Erweiterte Notification*. Statt nur „Akku noch X h Y min“ steht in der Sperrbildschirm-Anzeige jetzt zusätzlich die Confidence-Pille – die Vorhersage hat eine Verlässlichkeits-

Aussage immer mit dabei.

Die Refactor-Entscheidung folgt explizit den etablierten Mobile- Onboarding- und Explainable-AI-Best-Practices: drei Slides statt fünf (App-Onboarding-Studien zeigen einen deutlichen Abfall der Completion-Rate ab dem fünften Schritt), Confidence als visueller Indikator statt nackter Zahl, das eigentliche Feature direkt auf der Hauptbühne und Infrastruktur-Knöpfe in einem separaten Settings-Screen.

1.5 Python-Pipeline im Detail

Die Auswerte-Seite ist nach *Forschungs-Methodik* organisiert (eigener Ordner pro Methode, separater Eval-Ordner). Der Orchestrator `run_pipeline.py` bringt sie in Reihenfolge:

1. `methods/tinyml/merge_devices.py` – alle Geräte-CSVs zu einer kombinierten CSV zusammenführen
2. `methods/tinyml/data_prep.py` – Discharge-Segmente finden, Sliding-Windows bilden, Segment-Level-Split
3. `methods/tinyml/train.py` – Conv1D-Modell trainieren
4. `methods/tinyml/tflite_convert.py` – Konvertierung in drei TFLite-Varianten (dynamic, float16, int8)
5. `methods/*/predict.py` – Vorhersagen aller sechs verglichenen Methoden auf dem Test-Set
6. `evaluation/*.py` – diverse Auswertungs-Skripte

2 Datenmodell

2.1 CSV-Schema

Pro Messpunkt schreibt die App eine Zeile mit folgenden Spalten:

Tabelle 3: Felder eines CSV-Eintrags

Spalte	Typ	Bedeutung
<code>session_id</code>	String (8)	Zufalls-ID pro Service-Start
<code>timestamp</code>	long (ms)	Unix-Zeitstempel
<code>datetime</code>	String	YYYY-MM-DD HH:MM:SS
<code>battery_level</code>	Float 0–100	Akku-Stand in Prozent
<code>screen_on</code>	Int 0/1	Bildschirm an?
<code>brightness</code>	Float 0–100	Helligkeit normalisiert
<code>active_app_category</code>	Int 0–5	0=idle, 1=social, 2=video, 3=gaming, 4=browser, 5=productivity
<code>wifi_on</code>	Int 0/1	WiFi-verbunden?
<code>mobile_data_on</code>	Int 0/1	Mobilfunk aktiv?
<code>charging</code>	Int 0/1	Lädt gerade?
<code>cpu_usage</code>	Float 0–100	CPU-Frequenz-Auslastung (Proxy)
<code>temperature</code>	Float (°C)	Akku-Temperatur
<code>hotspot_on</code>	Int 0/1	WLAN-Hotspot aktiv?
<code>system_estimate_min</code>	Float	Google-API-Schätzung in Minuten
<code>system_personalized</code>	Int –1/0/1	Ist Google-Schätzung gerätspezifisch?
<code>own_prediction_h</code>	Float	Eigene TinyML-Vorhersage (Stunden)
<code>linear_baseline_h</code>	Float	Naive Vorhersage aus BatteryManager-Countern

Die ersten drei Spalten sind Metadaten, die zehn nächsten sind die *Features* für das ML-Modell, die letzten vier sind Vergleichswerte zur Auswertung.

2.2 Warum genau diese zehn Features?

Bei der Auswahl gab es eine harte Regel: *ich darf nur lesen, was eine normale App ohne Spezial-Berechtigung lesen kann*. Damit fielen einige naheliegende Kandidaten weg:

- **Genaue Stromstärke pro App** (PowerStats HAL): nur System-Apps.
- **Battery Health** (Verschleißzustand): teilweise gesperrt.
- **Kernel-Fuel-Gauge**: gesperrt.

Was übrig blieb, sind die zehn Features oben. Sie decken drei Gruppen ab: **Akkuzustand**

(level, temperature, charging), **Nutzungsintensität** (screen, brightness, app_category, cpu_usage), **Konnektivität** (wifi, mobile, hotspot).

2.3 Speicherung und Export

Die CSV liegt im app-internen Verzeichnis (`Context.filesDir`). Das ist nur für diese App lesbar – weder andere Apps noch ein angeschlossener PC können die Datei direkt sehen (siehe Datenschutz-Subsection 2.4). Der Export erfolgt explizit per Share-Intent durch den Nutzer.

Über vier Geräte hinweg wurden in 45 Tagen **66.001 Messpunkte** in **180 Sessions** gesammelt.

2.4 Datenschutz

Was wird erhoben

Die App speichert ausschließlich technische Geräte-Metriken (die Spalten oben). Es gibt *keinen* Zugriff auf:

- Standort (keine `ACCESS_*_LOCATION`-Permission),
- Kontakte, Anrufprotokolle, SMS,
- Mikrofon, Kamera,
- Inhalte von Apps (kein Accessibility-Service, kein Screen-Recording),
- personenbezogene Identifikatoren wie IMEI oder Telefonnummer.

Das einzige potenziell sensible Feld ist `active_app_category`, und auch das nur als *Kategorie* (0–5), nicht als konkreter Paketname.

Wo gespeichert wird

Im internen Verzeichnis der App (`Context.filesDir`). Nur diese App selbst hat Zugriff; andere Apps können nichts lesen (Android-Sandbox); bei Deinstallation werden die Daten automatisch mit entfernt; die Datei wird weder in iCloud noch in Google Backup synchronisiert. Ein Export findet ausschließlich über expliziten Nutzerklick (Share-Intent) statt – es gibt keinen automatischen Upload, keinen Server und keine Telemetrie.

Berechtigungen

Tabelle 4: Im Manifest deklarierte Permissions

Permission	Wofür
FOREGROUND_SERVICE	Background-Service mit Notification
FOREGROUND_SERVICE_SPECIAL_USE	Service-Typ ab Android 14
WAKE_LOCK	Service läuft auch bei Display aus
RECEIVE_BOOT_COMPLETED	Auto-Restart nach Geräte-Neustart
ACCESS_NETWORK_STATE	WiFi-/Mobile-Daten-Status lesen
ACCESS_WIFI_STATE	WiFi-Status, Hotspot-Reflection
PACKAGE_USAGE_STATS	Aktive App lesen (Nutzer muss manuell aktivieren)
POST_NOTIFICATIONS	Vorhersage in Notification anzeigen
REQUEST_IGNORE_BATTERY_OPTIMIZATIONS	Service vor Doze schützen

Was der Nutzer aktiv freischalten muss. Drei dieser Permissions kann der Nutzer Android nicht einfach durchwinken – sie brauchen ein explizites Antippen, weil sie weit mehr können als die App tatsächlich braucht. Genau das ist auch ein wichtiger Datenschutz-Mechanismus von Android. Die App führt den Nutzer per Button-Klick direkt in die jeweils richtige Systemeinstellung:

Tabelle 5: Berechtigungen mit Nutzer-Aktion

Berechtigung	Wie wird sie erteilt?	Warum braucht die App sie?
Benachrichtigungen	Standard-Dialog beim ersten Start (ab Android 13)	Foreground-Service braucht Notification, über die er sichtbar ist; ohne POST_NOTIFICATIONS startet der Service gar nicht.
Nutzungsdatenzugriff	Manuell in den Systemeinstellungen via Button „Nutzungsdatenzugriff aktivieren“	Damit die App weiss, welche App-Kategorie gerade aktiv ist (Social, Video, Spiel, ...) – eines der zehn Features. Ohne Aktivierung fällt das Feature einfach auf „Leerlauf“ zurück und die App läuft trotzdem.
Akku-Optimierung deaktivieren	Standard-Dialog via Button „Akku-Optimierung deaktivieren“	Damit der Service auch nach Stunden Inaktivität nicht von Doze oder herstellerseitigen Aggressive-Killer-Regeln (vor allem MIUI) unterbrochen wird. Auf Pixel-Geräten weniger kritisch.

Wo der Nutzer das findet. Alle drei Buttons sind im *Settings*-Screen (Zahnrad-Icon oben rechts) gebündelt – in der Sektion „Berechtigungen“ mit den Beschriftungen „Nutzungsdatenzugriff aktivieren“ und „Akku-Optimierung deaktivieren“. Ein Tap genügt, dann landet der Nutzer direkt auf der passenden System-Einstellung (`ACTION_USAGE_ACCESS_SETTINGS` bzw. `ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS`). Der Status „erteilt“ bzw. „fehlt“ wird im Settings-Screen direkt darüber angezeigt.

Beim Verteilen in der Familie

Für die drei Pixel-Geräte habe ich die App-APK an drei Familienmitglieder weitergegeben (per E-Mail bzw. direkt am Phone installiert). Vor der Installation wurde jeweils erklärt, welche Daten gesammelt werden (CSV mit 17 Spalten, davon 10 Sensor-Features plus Metadaten und Vergleichswerte), dass sie nur lokal auf ihrem Phone liegen, dass die CSV nur zu mir kommt, wenn sie sie aktiv per Share-Intent verschicken, und dass keinerlei Auto-Upload oder Hintergrund-Übertragung stattfindet. Eine Einwilligung hat jeder per WhatsApp bestätigt.

3 Programmierung

3.1 Datensammlung im Service

Der Kern des DataCollectorService ist ein Handler.postDelayed-Loop, der alle 30 Sekunden eine Messung auslöst:

```

1  private val INTERVAL_MS = 30 * 1000L // 30 Sekunden
2
3  private val collectRunnable = object : Runnable {
4      override fun run() {
5          collectAndLog()
6          handler.postDelayed(this, INTERVAL_MS)
7      }
8  }
9
10 private fun collectAndLog() {
11     // 1) Roh-Features lesen
12     val batteryLevel = getBatteryLevel()
13     val screenOn = if (isScreenOn()) 1 else 0
14     val brightness = getScreenBrightness()
15     // ... (10 Features insgesamt)
16
17     // 2) TinyML-Vorhersage (fail-safe)
18     val features = floatArrayOf(batteryLevel, screenOn.toFloat(), /*
19         ... */
20     var prediction = -1f
21     try {
22         predictor?.let { prediction = it.addDataAndPredict(features)
23     } catch (e: Exception) {
24         Log.e("BatteryCollector", "Prediction failed: ${e.message}")
25     }
26
27     // 3) Vergleichswerte einlesen
28     val systemEstimateMin = getSystemBatteryEstimate()
29     val linearBaselineH = getLinearBaselineHours()
30
31     // 4) Datenpunkt schreiben
32     logger.log(BatteryDataPoint(/* ... */), sessionId)

```

Quellcode 1: Mess-Schleife im DataCollectorService

3.2 Auslesen der 10 Features

Die meisten Werte kommen direkt aus den Android-System-Services. Drei Sachen waren tricky.

CPU-Auslastung

Seit Android 8 ist `/proc/stat` für Apps gesperrt. Statt der „echten“ CPU-Auslastung lese ich die aktuelle CPU-Frequenz pro Kern und vergleiche mit der Maximalfrequenz. Höhere Frequenz = höhere Last:

```
1 private fun getAvgCpuFrequencyPercent(): Float {
2     val numCores = Runtime.getRuntime().availableProcessors()
3     var totalRatio = 0f
4     var readCores = 0
5     for (i in 0 until numCores) {
6         try {
7             val cur = File("/sys/devices/system/cpu/cpu$i/cpufreq/
              scaling_cur_freq")
8                 .readText().trim().toLong()
9             val max = File("/sys/devices/system/cpu/cpu$i/cpufreq/
              cpuinfo_max_freq")
10                .readText().trim().toLong()
11             if (max > 0) {
12                 totalRatio += cur.toFloat() / max
13                 readCores++
14             }
15         } catch (_: Exception) { }
16     }
17     if (readCores == 0) throw Exception("No CPU freq readable")
18     return (totalRatio / readCores * 100f).coerceIn(0f, 100f)
19 }
```

Quellcode 2: CPU-Last als Frequenz-Proxy

Aktive App

Mit dem `UsageStatsManager` (Permission `PACKAGE_USAGE_STATS`, die der Nutzer manuell in den Einstellungen aktivieren muss) lese ich die zuletzt benutzte App und ordne sie über ein Package-Prefix-Mapping einer Kategorie zu:

```
1 private val APP_CATEGORIES = mapOf(
2     "com.instagram"           to 1, // social
3     "com.whatsapp"            to 1,
4     "org.telegram"            to 1,
5     "com.google.android.youtube" to 2, // video
6     "com.netflix"             to 2,
```

```

7      "com.spotify"                to 2,
8      "com.supercell"              to 3, // gaming
9      "com.miHoYo"                 to 3,
10     "com.google.android.apps.chrome" to 4, // browser
11     "org.mozilla"                 to 4,
12     "com.google.android.apps.docs"  to 5, // productivity
13     "com.microsoft"                to 5,
14     // ... (rund 30 Eintraege)
15 )

```

Quellcode 3: App-Kategorisierung (Auszug)

Unbekannte Packages bekommen Kategorie 0 (idle), werden aber zusätzlich geloggt, damit ich sie später nachmappen kann.

Hotspot-Status

Die offizielle `WifiManager.isWifiApEnabled()`-API ist als `@hide` markiert, also nicht im SDK. Über Reflection ist sie trotzdem erreichbar:

```

1 private fun isHotspotOn(): Boolean {
2     return try {
3         val wm = applicationContext.getSystemService(Context.
4             WIFI_SERVICE) as WifiManager
5         val method = wm.javaClass.getDeclaredMethod("isWifiApEnabled")
6         method.isAccessible = true
7         method.invoke(wm) as Boolean
8     } catch (e: Exception) {
9         false
10    }
11 }

```

Quellcode 4: Hotspot-Status via Reflection

3.3 On-Device-Inferenz mit TFLite

Der `BatteryPredictor` hält das TFLite-Modell und einen Ringpuffer mit den letzten 10 Datenpunkten. Erst wenn der Puffer voll ist, wird vorhergesagt. Wichtig: die Features müssen mit denselben `StandardScaler`-Werten normalisiert werden, mit denen das Modell trainiert wurde – die Werte sind hardcoded in der App und werden bei jedem Re-Training neu hineingeschrieben.

```

1 class BatteryPredictor(context: Context) {
2     companion object {
3         const val SEQUENCE_LENGTH = 10

```



```

4      const val NUM_FEATURES = 10
5      // Werte aus models/scaler.joblib (Multi-Device-Training)
6      private val SCALER_MEAN = floatArrayOf(
7          68.79f, 0.83f, 8.47f, 1.31f, 0.08f,
8          0.92f, 0.00f, 46.35f, 33.00f, 0.07f
9      )
10     private val SCALER_SCALE = floatArrayOf(
11         29.61f, 0.38f, 13.01f, 1.17f, 0.27f,
12         0.27f, 1.00f, 16.93f, 4.21f, 0.26f
13     )
14 }
15 private val interpreter: Interpreter
16 private val recentData = ArrayDeque<FloatArray>(SEQUENCE_LENGTH)
17
18 fun addDataAndPredict(features: FloatArray): Float {
19     // NaN/Inf-Guard vor StandardScaler
20     for (v in features) if (v.isNaN() || v.isInfinite()) return
21         -1f
22
23     val normalized = FloatArray(NUM_FEATURES) { i ->
24         val s = SCALER_SCALE[i]
25         if (s > 1e-6f) (features[i] - SCALER_MEAN[i]) / s else 0f
26     }
27     if (recentData.size >= SEQUENCE_LENGTH) recentData.
28         removeFirst()
29     recentData.addLast(normalized)
30     if (recentData.size < SEQUENCE_LENGTH) return -1f
31
32     val input = ByteBuffer.allocateDirect(SEQUENCE_LENGTH *
33         NUM_FEATURES * 4)
34         .order(ByteOrder.nativeOrder())
35     for (step in recentData) for (v in step) input.putFloat(v)
36     val output = ByteBuffer.allocateDirect(4).order(ByteOrder.
37         nativeOrder())
38     interpreter.run(input, output)
39     output.rewind()
40     return output.float.coerceAtLeast(0f)
41 }

```

Quellcode 5: Inferenz mit TFLite

3.4 Robustheit – die App muss überleben

Das war der frustrierendste Teil. Android killt Hintergrund-Services aggressiv, gerade auf Xiaomi-Geräten mit MIUI-Aufsatz. Drei Mechanismen sorgen dafür, dass die Datensamm-

lung nicht abreißt:

1. **Foreground-Service** mit dauerhafter Notification.
2. **Auto-Restart nach Wegwischen** der App-Karte. Wenn der Nutzer die App in der Übersicht wegwischt, registriert sich der Service über `AlarmManager.setExactAndAllowWhileIdle` einen Restart in 1 Sekunde.
3. **Auto-Restart nach Geräte-Neustart** über einen `BootReceiver`, der das `BOOT_COMPLETED`-Intent abfängt – aber nur, wenn der Nutzer den Sammler explizit aktiviert hatte.

```

1  override fun onTaskRemoved(rootIntent: Intent?) {
2      val restartIntent = Intent(this, DataCollectorService::class.java
3      )
4      val pendingIntent = PendingIntent.getService(
5          this, 99, restartIntent,
6          PendingIntent.FLAG_ONE_SHOT or PendingIntent.FLAG_IMMUTABLE
7      )
8      val alarm = getSystemService(Context.ALARM_SERVICE) as
9      AlarmManager
10     alarm.setExactAndAllowWhileIdle(
11         AlarmManager.ELAPSED_REALTIME_WAKEUP,
12         SystemClock.elapsedRealtime() + 1000,
13         pendingIntent
14     )
15     super.onTaskRemoved(rootIntent)
16 }

```

Quellcode 6: Auto-Restart bei onTaskRemoved

3.5 Schema-Drift im Logger

Beim Hinzufügen neuer Spalten (etwa `system_personalized` oder `linear_baseline_h`) hatten bestehende CSV-Dateien weniger Spalten als die neue Version erwartete. Der Logger prüft deshalb beim Start die Header-Zeile und parkt alte CSVs automatisch:

```

1  init {
2      val expectedHeader = "session_id,timestamp,datetime,{
3      BatteryDataPoint.CSV_HEADER}"
4      if (!csvFile.exists() || csvFile.length() == 0L) {
5          csvFile.writeText("$expectedHeader\n")
6      } else {
7          val firstLine = csvFile.useLines { it.firstOrNull() ?: "" }
8          if (firstLine != expectedHeader) {
9              val backup = File(context.filesDir,
10                  "battery_data_legacy_${System.currentTimeMillis()}.
11                  csv")

```

```

10         csvFile.copyTo(backup, overwrite = true)
11         csvFile.writeText("$expectedHeader\n")
12     }
13 }
14 }

```

Quellcode 7: Schema-Drift im Logger abfangen

3.6 Python-Pipeline – Beispiel Segment-Erkennung

Die wichtigste Pipeline-Funktion findet Entlade-*Segmente*: zusammenhängende Phasen, in denen das Phone entladen wird, ohne zwischendurch zu laden. Daraus werden später Sliding-Window-Sequenzen gebildet.

```

1 def find_discharge_segments(df, min_length, max_gap_ms):
2     segments = []
3     for _, session_df in df.groupby("session_id"):
4         session_df = session_df.sort_values("timestamp").reset_index(
5             drop=True)
6         is_discharging = session_df["charging"] == 0
7         block_id = (~is_discharging).cumsum() # neuer Block bei
8             Laden
9         for _, block in session_df[is_discharging].groupby(block_id[
10             is_discharging]):
11             # innerhalb des Blocks bei groesseren Zeitluecken
12             splitten
13             time_diffs = block["timestamp"].diff()
14             gap_id = (time_diffs > max_gap_ms).cumsum()
15             for _, sub in block.groupby(gap_id):
16                 if len(sub) >= min_length:
17                     segments.append(sub.reset_index(drop=True).copy())
18             )
19     return segments

```

Quellcode 8: Discharge-Segmente finden

3.7 Trainings-Verlauf

Abbildung 10 zeigt den Trainings- und Validierungs-Verlauf des Conv1D-Modells über die 21 Epochen, nach denen EarlyStopping greift.

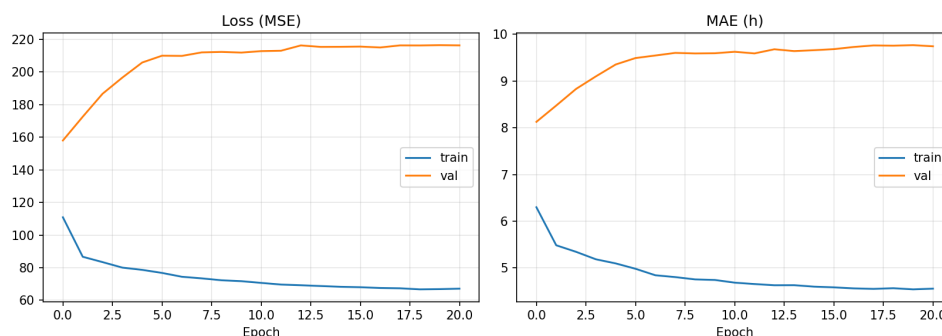


Abbildung 10: Trainings- und Validation-MAE des Conv1D-Modells. Auf dem Multi-Device-Datensatz erreicht das Modell nach 21 Epochen ein Validation-MAE von ca. 5 h. Da EarlyStopping mit `restore_best_weights=True` konfiguriert ist, wird das beste Modell zurückgegeben.

3.8 Code-Umfang und Eigenleistung

Über die sieben Bearbeitungswochen sind drei Schichten parallel entstanden – die Android-App in Kotlin, die ML-Pipeline am PC und die Auswertungs-Schicht. Tabelle 6 zeigt den Umfang nach Bereich; die wichtigsten Brocken sind die Auswertungs-Schicht (≈ 2.050 Zeilen reine Auswertungs-Logik) und die Android-App (≈ 850 Zeilen Kotlin, die produktionsnah auf vier Geräten laufen).

Tabelle 6: Code-Umfang nach Bereich

Bereich	Zeilen (ohne Leerzeilen)
Android-App Kotlin (14 Dateien)	≈ 1.450
Android-App Ressourcen (XML-Layouts + Drawables)	≈ 650
Python-Pipeline (<code>methods/</code> , 13 Module)	≈ 1.150
Python-Auswertung (<code>evaluation/</code> , 12 Module)	≈ 2.050
Konfiguration und Orchestrator	≈ 300
Summe	≈ 5.600

Selbst geschrieben:

- Gesamte Android-App-Struktur (Activity, Service, BroadcastReceiver, FileProvider), inklusive `onTaskRemoved`-Auto-Restart und Schema-Drift-Detection.
- Auswahl und Implementierung der zehn Features (CPU-Frequenz-Proxy, App-Kategorie-Mapping, Hotspot-Reflection).
- Datenmodell, CSV-Schema und FileProvider-Konfiguration für den Share-Export.

- Pipeline-Architektur (sechs verglichene Methoden je eigenes Modul, Trennung von Methoden- und Auswertungs-Schicht).
- Konkrete Implementierung der analytischen Baselines (Linear, Exponential mit Fall-back und Cap gegen Divergenz).
- Segment-Level-Split als Anti-Leakage-Mechanismus.
- Auswertungs-Skripte: Genauigkeits-Metriken, Per-Device-Analyse, Daten-Diagnose.

Übernommen:

- Conv1D-Architektur in Anlehnung an gängige TinyML-Topologien [1, 3] (zwei Convolutions, GlobalAveragePooling, zwei Dense).
- TFLite-Konvertierungs-API (Standard-Funktionen von TensorFlow).
- `scikit-learn`: `StandardScaler`, `RandomForestRegressor` [4], `train_test_split`.
- `scipy`: `curve_fit` für den exponentiellen Fit.

Probleme und wie sie gelöst wurden:

- *Service stirbt auf MIUI*: Auto-Restart über AlarmManager.
- */proc/stat gesperrt*: Frequenz-Proxy statt echter Auslastung.
- *CSV-Schema-Migration*: Header-Check beim Service-Start.
- *TinyML auf Single-Device zeigt MAE 0.5h Artefakt*: Segment-Level-Split eingeführt, korrigiert den scheinbaren Wert auf 9.96 h.
- *Exponential-Fit divergiert manchmal in 30.000h*: 72-h-Cap als Sanity-Bound.

3.9 Reproduzierbarkeit der Pipeline

Damit die Auswertung nicht von Hand-Schritten abhängt, gibt es einen einzigen Orchestrator `run_pipeline.py`, der die komplette Verarbeitungskette ausführt – vom Mergen der Geräte-CSVs bis zu den fertigen Auswertungs-Reports.

```
1 # vollstaendige Pipeline: ca. 20 Minuten auf Notebook-CPU
2 python run_pipeline.py
3
4 # nur die ML-Schritte (ohne Eval) -- ca. 5 Minuten
5 python run_pipeline.py --stage tinyml
```

Quellcode 9: Pipeline reproduzierbar starten

Alle Hyperparameter, Feature-Listen, Random-Seeds und Pfade liegen in `configs/default.yaml`. Beim Re-Training werden zusätzlich die `StandardScaler`-Werte für die Android-App neu exportiert (`methods/tinyml/export_scaler_for_android.py`) und ein Check-Script vergleicht sie mit den hardcoded Werten in `BatteryPredictor.kt`, damit die Normalisierung in App und Modell garantiert übereinstimmt.

4 Abschließender Erfahrungsbericht

4.1 Wöchentliche Einträge

Woche 1 (KW 13: 23.–29. März)

Setup und erste Datensammlung. Direkt nach der Auftakt-Sitzung der Vertiefung das Repo angelegt und das Android-Projekt aufgesetzt. Erste Version des `DataCollectorService` noch während der Vorlesungs-Stunde zu Foreground-Services live mit Claude geprototyp – so konnte ich die Service-Lifecycle-Theorie aus der Folie direkt am laufenden Code nachvollziehen. Schon am 25. März lief die erste Messung auf dem Xiaomi durch. Hauptproblem in dieser Woche: die `PACKAGE_USAGE_STATS`-Permission ist nicht via Standard-Dialog vergewbar – der Nutzer muss in den Einstellungen manuell ankreuzen. Ich habe deshalb einen Button in der App eingebaut, der direkt zum `ACTION_USAGE_ACCESS_SETTINGS` springt.

Woche 2 (KW 14: 30. März – 5. April)

Service-Robustheit – und parallel Statistik-Grundlagen aufgefrischt. Datensammlung bricht beim Wegwischen aus der App-Übersicht ab. Drei Tage damit verbracht herauszufinden, wie man auf Xiaomi-MIUI einen Service vor dem Aggressive-Killing schützt. Lösung: `onTaskRemoved` → `AlarmManager.setExactAndAllowWhileIdle` → Service kommt 1 Sekunde später wieder. Plus `BootReceiver` damit nach Neustart automatisch weitergesammelt wird.

Parallel zu den langen Debugging-Abenden habe ich einige Stunden investiert, um Statistik-Grundlagen aufzufrischen, die ich für die spätere Modell-Auswertung brauche: Cross-Validation, Train/Test/Validation-Splits, Data-Leakage-Mechanismen und Confounder-Erkennung. Zum damaligen Zeitpunkt fühlte sich das wie ein Umweg an. Eine Woche später zeigte sich, dass genau diese Auffrischung den entscheidenden Aha-Moment ermöglicht hat.

Woche 3 (KW 15: 6.–12. April)

Erstes Modell, erste Auswertung – erstes Erschrecken. Genug Daten zusammen (~11.000 Messpunkte), erstes Training durchgelaufen, MAE 0.53 h. Habe das stolz aufgeschrieben. Dann stutzig geworden: warum ist das so viel besser als die Google-API? Beim Nachprüfen festgestellt, dass das ein klassisches Data-Leakage-Artefakt vom Random-Split war – exakt das Muster, das ich eine Woche zuvor in den Statistik-Grundlagen wiederholt hatte. Pipeline auf Segment-Level-Split umgestellt → MAE wird zu 9.96 h. Das war der methodische Wendepunkt der Arbeit.

Diese Phase habe ich besonders eng mit Claude im Desktop-App-Modus gearbeitet: jede Pipeline-Änderung wurde gegen die alten Ergebnisse gegenchecked, mit gegenseitigem

Auditieren der Trainings-Logs. Am 12. April App-APK an ein Familienmitglied mit Pixel 7 Pro weitergegeben.

Woche 4 (KW 16: 13.–19. April)

Multi-Device wird real – und parallel Computer-Vision- und Embedded-ML-Stoff durchgegangen. APK an zwei weitere Familienmitglieder verteilt (Pixel 8 Pro, Pixel 9 Pro XL). Wichtig vor allem die Aufklärung darüber, was genau gesammelt wird – ich habe einen Mini-Datenschutz-Hinweis schriftlich geschickt und mündlich erklärt. Alle drei haben zugestimmt. Parallel das CSV-Schema erweitert (`system_personalized`, `linear_baseline_h`) und den Logger so gebaut, dass er bei Schema-Wechsel die alte Datei sicher parkt.

Diese Woche wollte ich früh genug verstehen, welche Stellschrauben es bei der späteren Modell-Konvertierung gibt. Habe deshalb einige Abende mit Computer-Vision- und Embedded-ML-Material gegründet, vor allem zu INT8-Quantisierung, Post-Training-Quantization und der Frage, was eigentlich auf Mobile passiert, wenn ein Float32-Modell auf INT8 gequetscht wird. Hat später den TFLite-Konvertierungs-Schritt deutlich beschleunigt – ich wusste schon, was per-channel- vs. per-tensor-Quantization bedeutet, bevor mir die TFLite-Doku das erzählen musste.

Woche 5 (KW 17: 20.–26. April)

Auswertungs-Tiefe. Pipeline so umgebaut, dass sie alle vier Geräte-CSVs konsolidiert und auswertet. Mean-Predictor und Random Forest als Sanity-Modelle hinzugefügt, weil ein „TinyML schlägt nichts“ wenig aussagekräftig wäre, ohne zu zeigen womit es überhaupt verglichen wird. Dazu noch die analytischen Baselines (Linear, Exponential) implementiert.

Diese Woche viel mit Claude im Sandbox-Container gearbeitet: zwei der drei Baseline-Implementierungen sind kleine, gut isolierte Skripte gewesen – ideal um sie im Auto-Approve-Modus laufen zu lassen, während ich parallel an der eigentlichen Auswertungs-Logik weitergearbeitet habe. Der Effekt ist spürbar gewesen: zwei parallele Threads statt einer linearen Sequenz.

Woche 6 (KW 18: 27. April – 3. Mai)

Auswertungs-Infrastruktur fertig stellen. Auswertungs-Skripte für Modell-Genauigkeit, Gerät-Vergleiche und Datenverteilungs-Diagnose in die Pipeline integriert. Zwischendurch die Xiaomi-Datensammlung beendet (28. April) – die finale Datenmenge stand fest: 66.001 Messpunkte. Der Wechsel von Single-Device auf Multi-Device zeigte spürbar klarere Unterschiede zwischen den Methoden als noch zuvor; das hat die Entscheidung bestätigt, die

Datensammlung über mehrere Geräte hinweg auszuweiten.

Woche 7 (KW 19/20: 4.–11. Mai)

Konsolidierung, UI-Refactor und Dokumentation. Pixel-Datensammlung beendet (10. Mai). Die Per-Device-Auswertung zeigte deutlich unterschiedliche Modell-Performance je nach Gerät; beim Hineinschauen in die Datenverteilung wurde der Grund sichtbar: auf dem Xiaomi war der Akku 88 % der Zeit über 75 %, also kaum Entlade-Dynamik zum Lernen vorhanden.

UI-Refactor (letzte Tage vor Abgabe). Der bisherige Single-Screen-Stand der App war für eine reine Datensammelungs-Phase ausreichend, für die Verteidigung aber zu schwach. In den letzten Tagen vor Abgabe deshalb noch ein überschaubarer aber inhaltlich entscheidender Refactor durch:

- Drei-Slide-Onboarding inklusive Modell-Größen-Vergleich als Wow-Moment beim ersten Start.
- Trennung der App in zwei klar getrennte Tabs (*Live* mit Vorhersage + Confidence-Pille + den 10 Live-Sensoren, *Genauigkeit* mit dem 3-Wege-Score zwischen eigenem Modell, Google und Linear-Baseline).
- Alle Infrastruktur-Buttons in eine separate *Settings*-Activity hinter ein Zahnrad-Icon ausgelagert, damit die Hauptbühne der App das eigene Hauptfeature trägt und kein Knopf-Buffer wird.
- Confidence-Anzeige sowohl im Live-Tab als auch in der Notification, damit der Vorhersage immer eine Verlässlichkeits-Information beiliegt.

Dieser Refactor lief vollständig im Sandbox-Container mit Claude im Auto-Approve-Modus: viele wiederholte XML-Layout-Edits, neue Fragments, neue Drawables. Genau die Art von Arbeit, die mit manueller Bestätigung pro Datei händisch erschöpfend gewesen wäre, im Container aber durchlaufen konnte, während ich parallel am Portfolio weitergeschrieben habe.

Eine ehrliche kleine Sackgasse zur Architektur-Visualisierung. Im Algorithmus-Tab zeigt die obere Schicht der Conv1D-Architektur eine 10×10-Matrix als Heatmap. Diese ist standardmäßig mit einem ruhigen Pseudo-Zufalls-Muster gefüllt, das die spätere Forward-Pass-Animation gut transportiert. Ich habe in den letzten Tagen vor Abgabe kurz überlegt, diese Matrix stattdessen mit den echten Sensor-Werten der letzten zehn Messpunkte zu füttern, um zu sagen: „auch der Input-Layer ist live“. Die Implementierung dafür war in 30 Minuten gebaut. Beim Testen auf dem Gerät stellte sich aber heraus: die meisten Sensoren ändern sich über fünf Minuten kaum (Akkustand schwankt

um wenige Prozent, Hotspot ist konstant aus, Ladezustand konstant). Die Matrix sah dadurch aus wie horizontale Streifen mit konstanten Werten – visuell schlechter als die ruhig pulsierende Pseudo- Heatmap und für den Betrachter ohne mündliche Erklärung auch nicht aussagekräftiger. Also habe ich die Live-Variante wieder verworfen und die abstrakte Architektur-Skizze beibehalten. Das Output-Element (die fertige Vorhersage) ist und bleibt ohnehin echt – nur die inneren Schichten sind bewusst als skizzierte Pulsations-Animation belassen, weil INT8-Aktivierungen visuell keinen Mehrwert hätten.

Den Rest der Woche viel Zeit in die Aufbereitung beider Abgabe- Dokumente (Portfolio und separates Paper) gesteckt, in Pipeline- Konsistenz-Checks und in das Schreiben dieses Erfahrungsberichts.

4.2 Schlüssel-Entscheidungen, Fehler und Trade-offs

Ein Teil der Glaubwürdigkeit dieser Arbeit liegt nicht in dem, was am Ende funktioniert, sondern in den Entscheidungen, die unterwegs gegen das Naheliegende getroffen wurden. Dieser Abschnitt macht die wichtigsten davon explizit – inklusive der Fehler, die mich dorthin geführt haben.

Die Story in vier Wendepunkten. Vier Momente haben den Verlauf des Projekts inhaltlich geprägt. Sie sind in Abbildung 11 chronologisch eingezeichnet.

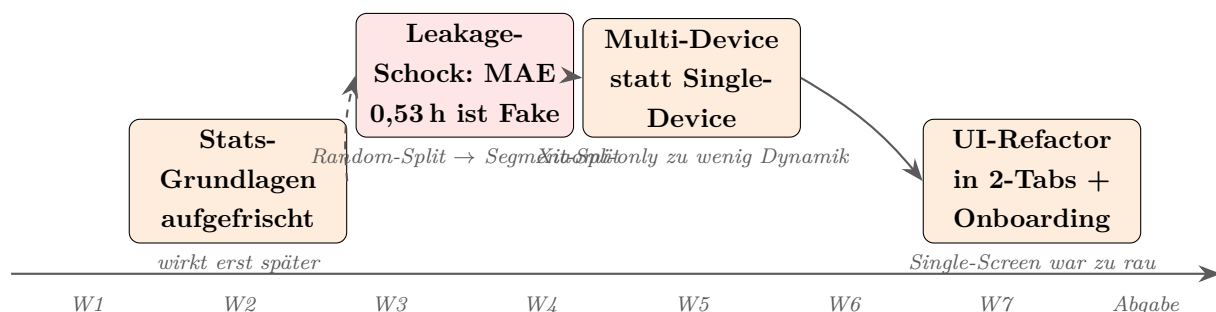


Abbildung 11: Die vier Wendepunkte der Projekt-Story. Orange: bewusste Entscheidung, Rot: vermiedener Irrtum. Gestrichelte Linien deuten an, dass sich der Stats-Aufwand in Woche 2 erst eine Woche später beim Leakage-Schock auszahlt.

Die wichtigsten Entscheidungen im Detail. Die folgende Tabelle macht acht zentrale Stell-Entscheidungen sichtbar, inklusive der jeweils verworfenen Alternative.

Tabelle 7: Schlüssel-Entscheidungen und verworfene Alternativen

Wann	Was gewählt	Alternative verworfen	Warum so
Konzeption	Eigene Daten + on-device Modell	Vorhandener Datensatz, Cloud-Modell	Liefert echten Real-World-Bezug und volle MLOps-Schleife
W1	10 Features pro Messpunkt	3–4 „sichere“ oder 20+	Kompromiss aus Vollständigkeit und Speicher-Overhead
W2	Statistik-Grundlagen parallel auffrischen	Erst loslegen, Theorie später	Hat den Aha-Moment in W3 erst möglich gemacht
W3	Segment-Level-Split	Random-Split (initial)	Random war Data-Leakage; MAE wurde von Fake-0,53 h zu echtem 9,96 h
W3–4	Multi-Device-Datensammlung	Nur Xiaomi (eigenes Gerät)	Xiaomi-Daten zu 88 % über 75 % – zu wenig Entlade-Dynamik
W5	INT8-Quantisierung (14,4 KB)	Float32-Modell (~ 50 KB)	TFLite-INT8 reicht in der Genauigkeit, ist deutlich kleiner
W7	Ein Hauptfeature stolz, 2 Tabs + Settings	Single-Screen mit allen Funktionen	Feature-Buffer verwirrt; klare Story besser für Verteidigung
W7	Feature-Importance-Anzeige verworfen	Bar-Chart der Sensor-Wichtigkeit	battery_level dominiert zu stark – wäre entlarvend statt sexy
W7	Architektur-Heatmap als skizzierte Pulsation	Input-Layer live mit echten Sensor-Werten füttern	Echte Werte änderten sich über 5 Min kaum – horizontale Streifen statt aussagekräftige Heatmap, visuell schlechter
Durchgängig	KI-Modus zweigeteilt	Nur Desktop-App-Modus	Container kapselt Risiko für massive Auto-Refactors

Was ich rückblickend anders gemacht hätte. Drei ehrliche Lerneffekte aus dem Verlauf – bewusst keine nachträgliche Glättung der Geschichte:

- *Früher Multi-Device.* Die Xiaomi-only-Phase in den ersten drei Wochen hat methodisch wenig gebracht. Hätte ich von Anfang an parallele Datensammlung auf mindestens zwei Geräten organisiert, wäre der Verteilungs-Aha-Moment (Akku zu 88 % über 75 %) zwei Wochen früher gekommen.
- *UI-Investition nicht so spät.* Der UI-Refactor in Woche 7 war für die Verteidigung essenziell, lief aber unter Zeitdruck. Eine „30-Minuten-UI-Pflege“ pro Woche hätte das verteilt und entspannter gemacht.
- *Statistik-Grundlagen früher.* Die in W2 aufgefrischten Grundlagen (Cross-Validation, Leakage) waren goldwert. Idealerweise hätte ich sie nicht aus Reflex in Woche 2 nachgeholt, sondern schon vor Projekt-Start dabeigehabt.

4.3 Gesamt-Aufwand

Tabelle 8: Zeitaufwand nach Woche

Woche	Hauptaufwand	Stunden
1	App-Grundgerüst + Permissions	~ 15
2	Service-Robustheit auf MIUI + Statistik-Auffrischung	~ 21
3	Trainings-Pipeline + Leakage-Aha-Moment	~ 20
4	Schema-Migration + APK-Verteilung + Embedded-ML-Grundlagen	~ 13
5	6-Wege-Vergleich + analytische Baselines	~ 18
6	Auswertungs-Skripte + Daten-Diagnose	~ 15
7	Portfolio/Paper schreiben + Audits + UI-Refactor	~ 33
Summe		≈ 135 h

4.4 KI-Nutzung

Während des Projekts habe ich **Claude Code** (Anthropic, Modell Opus 4.7 mit 1M-Token-Kontext) als Programmier-Assistent verwendet. Dieser Abschnitt ist bewusst transparent gehalten.

Wofür Claude genutzt wurde:

- *Code-Refactoring* – Umstrukturierung von einem monolithischen Trainings-Skript zu einer modularen Pipeline (`methods/`, `evaluation/`).
- *Boilerplate* – TFLite-Konvertierungs-Boilerplate, Argument-Parser, Standard-Plot-Code (matplotlib).
- *Recherche-Beschleunigung* – Suche nach relevanter Android- und TinyML-Literatur.

- *Auswertungs-Skripte* – Modell-Genauigkeit, Gerät-spezifische Auswertung und Daten-Diagnose nach meinen Vorgaben.
- *Schreibhilfe* – Strukturierung dieses Portfolios und des separaten Papers, Tabellen-Formatierung, Korrektur-Lesen.
- *Audit-Runden* – systematisches Aufspüren eigener Fehler (inkonsistente Zahlen, methodische Schwächen, Confounder-Verdacht).

Was ich selbst gemacht habe:

- Das gesamte Projekt-Konzept und die App-Idee.
- Auswahl der zehn Features (was eine normale App ohne Spezial-Rechte überhaupt lesen darf).
- Datensammlung auf den vier Geräten – inklusive App-Verteilung an drei Familienmitglieder.
- Alle methodischen Entscheidungen: Feature-Auswahl, Segment-Level-Split (Trainingsdaten-Aufbereitung), Auswahl der drei in der App angezeigten Vergleichswerte.
- Kritisches Hinterfragen der KI-Vorschläge, wenn etwas verdächtig wirkte. Im Verlauf wurden so mehrere eigene Fehler (z. B. falsche Autoren bei drei Quellen, zu plakative Aussagen) korrigiert.
- Alle Anpassungen am Android-Manifest, Permissions, geräte-spezifisches Debugging (MIUI-Service-Stirbt-Problem).

Setup und Sicherheits-Rahmen der KI-Nutzung. Die KI lief in zwei klar getrennten Modi, je nach Risiko der Aufgabe:

- *Desktop-App-Modus (mit manueller Bestätigung).* Verwendet für die meisten Aufgaben am Haupt-Code: Refactoring, Schreibhilfe, Code-Review, Audit-Runden. Jede Datei- und Shell-Aktion musste explizit bestätigt werden – der Mehr- Aufwand ist die Versicherung gegen versehentliche Schäden am Hauptprojekt.
- *Sandbox-Container mit Auto-Approve.* Verwendet für riskantere oder experimentelle Aufgaben (z. B. Aufsetzen unbekannter Pipelines, breite Code-Suchen auf fremden Repos). Hier lief Claude in einem Docker-Container mit eingeschränktem Datei- und Netzwerk-Zugriff – nur das jeweilige Projekt war sichtbar, und nur eine kleine Domain-Whitelist (Anthropic API, GitHub, PyPI, npm) erreichbar. So konnte die KI eigenständig arbeiten, ohne dass ein potenzieller Fehler auf das Host-System durchschlagen konnte.

Genutzt wurde **Claude Pro** (Subscription), nicht der API-Zugang. Der Subscription-Modus deckelt das Budget automatisch über Zeitfenster – ein zusätzlicher Schutz gegen unbeabsichtigte Endlos-Schleifen.

Bilanz. Realistisch eingeschätzt: ohne KI-Unterstützung hätte ich das Projekt in dieser Tiefe nicht in sieben Wochen geschafft. Die KI war besonders nützlich für Boilerplate und Recherche-Beschleunigung sowie für methodische Disziplin (Konsistenz-Checks über große Code-Mengen). Was die KI *nicht* ersetzt hat: die Entscheidung, was gemacht wird, das Verstehen der Ergebnisse, das Verteidigen von Trade-offs, und vor allem die handfeste Geräte-Logistik und das Debugging vor Ort.

4.5 Abschluss-Reflexion

Was funktioniert (Stand 11. Mai 2026):

- *Bewährung im Realbetrieb.* Die App lief 45 Tage am Stück auf vier verschiedenen Geräten unter realer Alltags- Nutzung – kein Crash, kein Daten-Verlust. Der Foreground-Service hat über Boot-Reboots und das versehentliche Wegwischen aus dem Recent-Apps-Menü hinweg selbständig wieder gestartet. Diese 180 ungeplant getesteten Restart-Pfade sind das eigentliche Funktions-Argument für eine App, die im Hintergrund laufen soll.
- *Drei unabhängige Tester.* Drei Familienmitglieder haben die App mehrere Wochen lang ohne Eingreifen genutzt – die App musste also bedienbar genug sein, um ohne Anleitung im Alltag zu funktionieren. Die UI mit Onboarding und 2-Tab-Layout (Live-Vorhersage + Genauigkeits-Tab) hat dieses Feldtest bestanden.
- *Modell-Performance.* TFLite-Modell mit 14,4 KB, Inferenz in $\sim 5 \mu\text{s}$ pro Vorhersage – auch auf den älteren Test-Geräten ohne spürbaren Energie-Overhead.
- *Datensammlung.* 66.001 Messpunkte, sauber per Share- Intent exportiert, mit dokumentierter CSV-Schnittstelle zur Auswertungs-Pipeline.
- *Reproduzierbarkeit.* Trainings- und Auswertungs- Pipeline läuft mit einem einzigen Befehl (`python run_pipeline.py`) – der gesamte Weg von Rohdaten zum fertigen TFLite-Modell ist transparent und nachvollziehbar.

Was noch nicht ideal ist:

- Die Scaler-Werte werden per Skript aus dem Python-Modell ins Kotlin kopiert – ein automatischer Build-Step wäre eleganter.
- Die Confidence-Anzeige in der App basiert aktuell auf einer einfachen Heuristik (Buffer-Größe + Akkustand). Eine echte Quantil-Ausgabe direkt aus dem Modell

wäre wissenschaftlich sauberer.

- Der Genauigkeits-Vergleich der drei Methoden (eigenes Modell vs. Google vs. Linear) im Genauigkeits-Tab basiert auf dem letzten Discharge-Lauf – ein längerer Verlauf über mehrere Tage wäre aussagekräftiger.

Was ich aus dem Projekt mitnehme:

- *Android im Hintergrund ist hart.* Theoretisch sind Foreground-Services eine Standard-Lösung. Praktisch küllt jeder Hersteller anders. Multi-Device-Test ist die einzige Möglichkeit, das wirklich zu prüfen.
- *Methodische Disziplin schlägt Modell-Komplexität.* Der Wechsel von Random-Split zu Segment-Level-Split hatte mehr Einfluss auf das Ergebnis als jedes Hyperparameter-Tuning.
- *Negative Ergebnisse ehrlich darstellen ist genau so wertvoll wie positive.* Anfangs schien das Projekt zu „funktionieren“ (MAE 0.5 h). Erst durch saubere Methodik kam heraus, dass dieser Wert ein Leakage-Artefakt war. Das ehrlich aufzuschreiben ist anstrengender, aber wissenschaftlich korrekter.

4.6 Grenzen der Arbeit und Ausblick

Wo die Arbeit ehrlicherweise an Grenzen stößt:

- *Wenige Test-Geräte.* Vier Geräte (ein Xiaomi, drei Pixel) reichen, um zu zeigen, dass die Pipeline funktioniert. Sie reichen aber nicht, um eine allgemeine Aussage wie „auf Xiaomi-Geräten arbeitet das Modell besser als auf Pixel“ belastbar zu treffen – dafür bräuchte es deutlich mehr Geräte pro Hersteller.
- *Beobachtungszeitraum.* 45 Tage zeigen das Akku-Verhalten im Frühjahr 2026. Saisonale Effekte (Kälte im Winter senkt die Akku-Kapazität messbar) sind nicht abgedeckt. Eine Erhebung über ein ganzes Jahr würde das Modell robuster machen.
- *Nutzungsmuster der Test-Personen.* Drei der vier Geräte gehören Familienmitgliedern mit ähnlichem Nutzungsverhalten (Erwachsene, durchschnittliche Smartphone-Nutzung). Ein Vergleich mit z.B. Jugendlichen (Gaming, dauerhaftes Streamen) oder Senioren (sehr wenig Nutzung) wäre aufschlussreich, war im Zeitrahmen aber nicht zu leisten.
- *Googles Vergleichswert ist eine Black-Box.* Die App vergleicht ihre eigene Vorhersage mit Googles API, aber wie Google intern rechnet, ist nicht öffentlich. Ein *fairer* Vergleich ist daher nur eingeschränkt möglich; man vergleicht Ergebnis gegen Ergebnis, nicht Methode gegen Methode.

Logische nächste Schritte:

- *Algorithmus-Ansicht in der App.* Aktuell zeigt die App nur die drei Vorhersage-Zahlen. Eine zusätzliche Ansicht würde live darstellen, welche der zehn Sensoren gerade den größten Einfluss auf die Vorhersage haben („Helligkeit zieht aktuell am stärksten“). Das macht den Algorithmus für den Nutzer nachvollziehbar – man sieht dem Modell beim Denken zu – und ist genau die Transparenz, die einer App mit ML-Komponente gut steht.
- *Automatisches Nachtrainieren.* Aktuell wird das Modell manuell auf dem PC neu trainiert. Sinnvoll wäre ein Mechanismus, der das z. B. einmal pro Woche im Hintergrund anstößt, sobald genug neue Daten vorliegen – so passt sich das Modell ohne Zutun an Änderungen im Nutzungsverhalten an.
- *Training direkt auf dem Handy.* Die nächste, technisch reizvolle Stufe wäre, das Lernen nicht mehr am PC zu machen, sondern jedes Handy lokal an seinem eigenen Verhalten lernen zu lassen – ohne dass Daten das Gerät jemals verlassen. Das ist konzeptionell anspruchsvoll, würde aber den Datenschutz-Vorteil perfekt machen.
- *Breitere Studie.* Mit der vorhandenen App wäre eine Erhebung mit 20–50 Teilnehmern technisch problemlos möglich. Was dann zusätzlich nötig wäre: eine kleine Datenschutz-Erklärung und ein verständliches Onboarding für die Probanden, sowie die Klärung des Datenschutz-Status mit der Hochschule.

Übergeordnetes Fazit zur Meta-Ebene. Das Projekt behauptet nicht, dass das eigene Modell objektiv besser ist als Googles. Es zeigt, *dass* und *wie* man als einzelner Student in sieben Wochen ein vollständiges System aus App, Datensammlung und Vorhersage-Modell aufbauen kann – und ehrlich auswertet, wo das eigene Modell besser und wo es schlechter abschneidet. Diese *Nachvollziehbarkeit jedes Zwischenschritts* ist der eigentliche Wert – nicht eine punktgenaue Genauigkeits-Zahl.

Literatur

- [1] Norah N. Alajlan and Dina M. Ibrahim. TinyML: Enabling of inference deep learning models on ultra-low-power IoT edge devices for AI applications. *Micromachines*, 13(6):851, 2022.
- [2] Android Open Source Project. `PowerManager.getBatteryDischargePrediction()`, 2021. Android API level 31 (Android 12).
- [3] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, et al. MLPerf Tiny benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [4] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.